

第6章: プロジェクトセットアップ — 書籍管理アプリの土台を作る

いよいよ実際のアプリ開発が始まります！この章では、書籍管理アプリの「土台」（プロジェクトの雛形、フォルダ構成、初期設定）を作ります。

この章で行うこと

ここまでの章で学んだ技術（TypeScript、React、Next.js、Supabase）を組み合わせて、実際のプロジェクトを作成します。料理に例えると、**材料を揃えて下ごしらえをする段階**です。包丁やまな板、調味料を準備しておくのと同じで、後でスムーズに料理（コード）が書けるように、まずは作業環境を整えます。

1. Next.js プロジェクトの作成 — `npx create-next-app` コマンド（プロジェクトの雛形を自動生成するコマンド）を実行
2. Tailwind CSS の理解 — Tailwind CSS（テールウィンドCSS：HTMLのクラス名でスタイルを直接指定する CSSフレームワーク）の使い方
3. フォルダ構成の設計 — どのファイルをどこに置くか、プロジェクト全体の設計図を決める
4. Supabase との接続設定 — アプリからデータベースに接続するための設定ファイルを作成
5. 型定義の作成 — 書籍データの「Book型」をTypeScriptで定義

この章を終えると、ブラウザで開発サーバー（Development Server：開発中にアプリを確認するためのローカルサーバー。`http://localhost:3000` でアクセスできる）にアクセスし、ヘッダー付きのトップページが表示される状態になります。

用語のミニ解説： - **ビルド（build）**：ソースコード（人間が書いたコード）を、ブラウザや Node.js が実行できる形式に変換する作業のこと。TypeScript を JavaScript に変換したり、複数ファイルを1つにまとめたりします。 - **devサーバ（開発サーバー）**：コードを書きながら動作確認するためのローカル（自分のPC上）で動くWebサーバー。ファイルを保存すると自動的にブラウザの表示が更新されます。 - **依存関係（dependencies）**：あなたのアプリが動くために必要な外部のライブラリ（React や Next.js など）のこと。`package.json` に一覧が書かれます。 - **環境変数（environment variable）**：アプリの外側からプログラムに渡す設定値。コードに直接書きたくない秘密情報（API キーなど）や、開発環境と本番環境で値を変えたい設定に使います。

ポイント： この章はコードを書く量が多いですが、一つ一つのファイルの役割を理解しながら進めてください。「なぜこのファイルが必要なのか」を意識すると、後の章で迷わなくなります。

目次

1. Next.js プロジェクトの作成

2. Tailwind CSS の基礎
3. 必要なパッケージのインストール
4. プロジェクト構成の作成
5. Supabase クライアントの設定
6. 型定義ファイルの作成
7. ルートレイアウトの設定
8. 共通ヘッダーコンポーネント
9. 開発サーバーの起動と動作確認
10. データフローの全体像

1. Next.js プロジェクトの作成

1.1 create-next-app コマンドの実行

ターミナル（コマンドプロンプト、PowerShell、または VS Code のターミナル）を開き、プロジェクトを作成したいディレクトリに移動してから、以下のコマンドを実行します。

▼ このコードがやること（先に日本語で）：Next.js アプリの「雛形（ひながた：最初の土台一式）」を1コマンドで自動生成します。 `npx` は「PCにインストールしないで一時的にツールを実行する」仕組みで、ここでは `create-next-app`（プロジェクト作成ツール）の最新版を呼び出しています。 `book-management` の部分が作られるフォルダ名なので、好きな名前に変えてもかまいません。各単語の意味は下のコメントを見てください。

```
# npx: ローカルにインストールしていないコマンドも一時的に実行できるツール
# create-next-app: Next.js プロジェクトの雛形を自動生成するパッケージ
# @latest: パッケージの「最新版」を使うという指定 (@を使ってバージョン指定)
# book-management: 作成するプロジェクトのフォルダ名（好きな名前に変えてもOK）
npx create-next-app@latest book-management
```

npx とは？ `npx` は npm 5.2 以降に同梱されているコマンドで、パッケージをグローバルインストールせずに一時的に実行できます。`create-next-app@latest` は「最新版の create-next-app を使う」という意味です。プロジェクト作成のためだけに使うコマンドを、PCに永久にインストールせずに済むので便利です。

npm と npx の違い: `npm install` はパッケージを「インストール」して使えるようにするコマンドで、`npx` はパッケージを「実行」するコマンドです。`create-next-app` はプロジェクト作成時に一度だけ使うので、インストールせずに `npx` で実行します。

コマンドを実行すると、いくつかの質問が対話形式で表示されます。以下のように回答してください。

1.2 各選択肢の詳細解説

```
Would you like to use TypeScript? ... Yes           # TypeScriptを使うか? → 使う
Would you like to use ESLint? ... Yes               # ESLint (コード品質チェック) を使う
か? → 使う
Would you like to use Tailwind CSS? ... Yes         # Tailwind CSS (スタイリング) を使う
か? → 使う
Would you like your code inside a `src/` directory? ... Yes # コードを src/ フォルダに入れる
か? → 入れる
Would you like to use App Router? (recommended) ... Yes   # App Router (新しいルーティン
グ) を使うか? → 使う
Would you like to use Turbopack for next dev? ... Yes      # Turbopack (高速ビルドツール)
を使うか? → 使う
Would you like to customize the import alias (@/* by default)? ... No # @/* のエイリアスを
カスタムするか? → しない
```

それぞれの選択肢が何を意味するのか、なぜその回答をするのかを詳しく見ていきましょう。

質問	回答	理由
TypeScript	Yes	型安全性により、変数や関数の引数に間違った値を渡すミスをコンパイル時に検出できます。特にチーム開発やアプリが大規模になったときに威力を発揮します。本チュートリアルでは、Book 型などを定義してデータの構造を明確にします。
ESLint	Yes	JavaScript/TypeScript のコードを静的解析し、バグの原因になりやすい書き方やスタイルの不統一を自動検出します。例えば、未使用の変数がある場合に警告を出してくれます。
Tailwind CSS	Yes	ユーティリティファーストの CSS フレームワークです。HTML (JSX) の中にクラス名を書くだけでスタイリングできるため、CSS ファイルを別途管理する必要がありません。詳しくは次のセクションで解説します。
<code>src/</code> directory	Yes	ソースコードを <code>src/</code> ディレクトリにまとめることで、設定ファイル (<code>package.json</code> , <code>next.config.ts</code> など) とソースコードが明確に分離されます。プロジェクトのルートがすっきりし、見通しがよくなります。
App Router	Yes	Next.js 13 以降で導入された新しいルーティング方式です。ファイルシステムベースのルーティング、Server Components、レイアウトのネストなど、モダンな機能が使えます。Pages Router (旧方式) より推奨されています。
Turbopack	Yes	Rust で書かれた高速なバンドラーです。開発サーバーの起動やホットリロード (コード変更時の自動反映) が従来の Webpack より大幅に高速になります。
Import alias	No (デフォルト)	デフォルトの <code>@/*</code> エイリアスをそのまま使います。 <code>@/components/Header</code> のように <code>src/</code> ディレクトリ内のファイルを短いパスでインポートできます。

静的解析（static analysis）とは？ コードを実行せずに、ソースコードを読み解いて問題を見つける手法のこと。ESLint がこの役割を担います。例えば「この変数は宣言されているけど使われていない」「 `if` 文の条件が常に `true` になる」といった問題を、実行前にチェックしてくれます。

パスエイリアスとは？（重要） 通常、別のファイルをインポートするときは相対パス

（ `../../components/Header` のように、現在のファイルからの相対位置）を使います。深い階層になると `../../../../../` のように `../` が増えて読みづらくなります。

パスエイリアスを使うと、 `@/components/Header` のように `src/` を起点とした絶対インポート（プロジェクトのルートからのパス）で書けるようになります。ファイルを移動してもインポート文を直さなくてよいので、リファクタリング（コードの整理）も楽になります。

1.3 なぜ Tailwind CSS を選ぶのか

CSS のスタイリング手法にはさまざまな選択肢があります。以下の比較表で、それぞれの特徴を整理します。

項目	Tailwind CSS	CSS Modules	styled-components
スタイルの書き場所	JSX のクラス名に直接記述	<code>.module.css</code> ファイルに記述	JavaScript ファイル内に記述
学習コスト	クラス名を覚える必要がある（慣れると高速）	通常の CSS と同じ知識で書ける	CSS + JavaScript の知識が必要
ファイル数	CSS ファイル不要（少ない）	コンポーネントごとに CSS ファイルが必要（多い）	CSS ファイル不要（少ない）
名前の衝突	なし（ユーティリティクラスのため）	なし（自動でスコープされる）	なし（自動でスコープされる）
バンドルサイズ	未使用クラスが自動削除される（小さい）	使った分だけ（中程度）	ランタイムで CSS を生成（やや大きい）
レスポンシブ対応	<code>md:</code> , <code>lg:</code> などのプレフィックスで簡単	メディアクエリを自分で書く	メディアクエリを自分で書く
デザインの一貫性	デフォルトのデザインシステム内蔵	自分で設計する必要がある	自分で設計する必要がある
Server Components 対応	完全対応	完全対応	対応に制限あり（ <code>'use client'</code> が必要）
本チュートリアルでの採用	採用	不採用	不採用

Tailwind CSS を選ぶ最大の理由は「開発速度」です。CSS ファイルを作成して行き来する必要がなく、JSX を書きながらそのままスタイリングできます。最初はクラス名を覚えるのに少し時間がかかりますが、慣れると非常に高速に UI を構築でき

るようになります。

1.4 コマンド実行中のターミナル出力

質問に全て答え終わると、ターミナルに次のような表示が流れます（実際の表示は時期によって少し変わります）。

```
Creating a new Next.js app in /Users/you/projects/book-management.

Using npm.

Initializing project with template: app

Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- typescript
- @types/node
- @types/react
- @types/react-dom
- tailwindcss
- postcss
- eslint
- eslint-config-next

added 372 packages, and audited 373 packages in 24s

131 packages are looking for funding
found 0 vulnerabilities
Initialized a git repository.

Success! Created book-management at /Users/you/projects/book-management
```

「added XXX packages」とは?: `npm install` が `node_modules/` フォルダに何個のパッケージを置いたかを示します。React/Next.js本体に加えて、依存パッケージ（依存関係でついてくる関連ライブラリ）と一緒に入るので300〜400個程度になります。これが普通です。

`npm install` で何が起きているか: 1. `package.json` に書かれた依存パッケージのリストを読み取る 2. npm レジストリ (<https://www.npmjs.com>) から、それぞれのパッケージをダウンロード 3. ダウンロードしたパッケージを `node_modules/` フォルダに展開 4. 各パッケージがさらに依存しているパッケージも芋づる式にダウンロード（これが「依存関係の解決」） 5. インストールされた正確なバージョンを `package-lock.json` に記録

`node_modules/` はとても巨大になりやすいですが、`.gitignore` でGit管理から除外するのが標準的なやり方です（後述）。

dependencies と devDependencies の違い: - **dependencies**: アプリが本番環境で動くために必要なパッケージ。例: React、Next.js本体。 - **devDependencies**: 開発中だけ必要なパッケージ。例: TypeScript（本番ではJavaScriptに変換済み）、ESLint（本番では使わない）。

本番ビルド時には **devDependencies** を除外できるので、本番のサーバーに置くファイルを軽くできます。

完了したら、表示されたプロジェクトフォルダに `cd` で移動します。

```
# cd: Change Directory の略。指定したフォルダに移動するコマンド
# book-management: 移動先のフォルダ名（先ほど create-next-app で作成したフォルダ）
cd book-management
```

1.5 生成されるファイルの役割

`create-next-app` を実行すると、以下のファイルとディレクトリが自動生成されます。

book-management/	
node_modules/	... インストールされたパッケージ群（数百～数千フォルダ。 <code>.gitignore</code> で除外する）
public/	... 静的ファイル（画像など）。ここに置いたファイルは / 直下のURLでアクセス可能
file.svg	... ファイルアイコン（デフォルトサンプル）
globe.svg	... 地球アイコン（デフォルトサンプル）
next.svg	... Next.js ロゴ（デフォルトサンプル）
vercel.svg	... Vercel ロゴ（デフォルトサンプル）
window.svg	... ウィンドウアイコン（デフォルトサンプル）
src/	... アプリのソースコードを置くフォルダ
app/	... App Router のルートディレクトリ。フォルダ構成がそのままURLになる
favicon.ico	... ブラウザのタブに表示されるアイコン
globals.css	... グローバル CSS（Tailwind の設定を含む）
layout.tsx	... ルートレイアウト（全ページ共通の枠）
page.tsx	... トップページ（ / にアクセスしたときの表示）
.eslintrc.json	... ESLint の設定ファイル
.gitignore	... Git で管理しないファイルの指定
next-env.d.ts	... Next.js の TypeScript 型定義（自動生成）
next.config.ts	... Next.js の設定ファイル
package-lock.json	... パッケージの正確なバージョン記録（自動更新されるので手で編集しない）
package.json	... プロジェクトの設定とパッケージ一覧
postcss.config.mjs	... PostCSS の設定（Tailwind CSS が使用）
README.md	... プロジェクトの説明
tailwind.config.ts	... Tailwind CSS のカスタマイズ設定
tsconfig.json	... TypeScript の設定ファイル

各ファイルの役割を表にまとめます。

ファイル	役割
<code>package.json</code>	プロジェクト名、使用パッケージ、実行スクリプトなどを定義するファイルです。 <code>npm install</code> はこのファイルを読んでパッケージをインストールします。
<code>tsconfig.json</code>	TypeScript コンパイラの設定ファイルです。 <code>@/*</code> のパスエイリアスもここで定義されています。
<code>next.config.ts</code>	Next.js の動作をカスタマイズする設定ファイルです。画像の外部ドメイン許可などを設定できます。
<code>tailwind.config.ts</code>	Tailwind CSS のカスタマイズ設定ファイルです。カスタムカラーやフォントの追加ができます。
<code>postcss.config.mjs</code>	PostCSS（CSS の変換ツール）の設定です。Tailwind CSS は PostCSS のプラグインとして動作します。
<code>.eslintrc.json</code>	ESLint のルール設定です。 <code>next/core-web-vitals</code> の推奨ルールがデフォルトで有効になっています。
<code>src/app/layout.tsx</code>	すべてのページを囲むルートレイアウトです。 <code><html></code> と <code><body></code> タグを含み、共通のヘッダーやフッターを配置します。
<code>src/app/page.tsx</code>	トップページのコンポーネントです。URL <code>/</code> にアクセスしたときに表示されます。
<code>src/app/global.css</code>	アプリ全体に適用されるグローバル CSS です。Tailwind CSS のディレクティブ（ <code>@tailwind base;</code> など）が含まれています。

.gitignore で何を除外するか、なぜか: Git の管理から除外する（コミットしない）べきファイルの種類を **.gitignore** ファイルに書きます。 **create-next-app** が自動生成する **.gitignore** には、次のようなものが含まれます。

除外対象	理由
<code>node_modules/</code>	パッケージ群はとて大きい上に <code>package.json</code> から再インストールできるため。コミットすると無駄に容量を食う。
<code>.next/</code>	Next.js のビルド結果。実行のたびに再生成されるため。
<code>.env*.local</code>	秘密の環境変数（APIキー、パスワードなど）が入っているため。GitHub にアップすると漏洩する。
<code>*.log</code> , <code>npm-debug.log*</code>	ログファイルは個人のPC固有のものなので共有不要。
<code>.DS_Store</code> , <code>Thumbs.db</code>	macOS/Windowsが自動生成する隠しファイル。アプリと無関係。

ルール: 「自動生成されるもの」「PC固有のもの」「秘密情報を含むもの」はGitに入れない、と覚えておくの良いです。

1.6 主な設定ファイルの中身を見る

`package.json` の構造

create-next-app が生成する `package.json` は、おおよそ次のような構造になっています。

```

{
  // プロジェクト名 (フォルダ名と同じ)
  "name": "book-management",
  // プロジェクトのバージョン (自由に決められる)
  "version": "0.1.0",
  // true にすると npm publish で公開できなくなる (誤公開防止)
  "private": true,
  // npm run <名前> で実行できるコマンドの集まり
  "scripts": {
    // 開発サーバー起動 (Turbopackを使用)
    "dev": "next dev --turbo",
    // 本番用ビルド (最適化された静的ファイルを生成)
    "build": "next build",
    // ビルド済みアプリを本番モードで起動
    "start": "next start",
    // ESLint でコード品質チェックを実行
    "lint": "next lint"
  },
  // 本番でも必要なパッケージ
  "dependencies": {
    // React 本体
    "react": "19.x.x",
    // React を DOM にレンダリングするためのパッケージ
    "react-dom": "19.x.x",
    // Next.js フレームワーク
    "next": "15.x.x"
  },
  // 開発中だけ必要なパッケージ
  "devDependencies": {
    // TypeScript コンパイラ
    "typescript": "^5",
    // Node.js の型定義
    "@types/node": "^20",
    // React の型定義
    "@types/react": "^19",
    // React DOM の型定義
    "@types/react-dom": "^19",
    // PostCSS (CSS変換ツール)
    "postcss": "^8",
    // Tailwind CSS 本体
    "tailwindcss": "^3.x.x",
    // ESLint 本体
    "eslint": "^8",
    // Next.js 用 ESLint 設定
    "eslint-config-next": "15.x.x"
  }
}

```

tsconfig.json の主要設定

```
{
  // TypeScript コンパイラの動作設定
  "compilerOptions": {
    // コンパイル後のJavaScriptのバージョン (ES2017相当)
    "target": "ES2017",
    // 使えるグローバルAPI (DOM、最新のES機能など)
    "lib": ["dom", "dom.iterable", "esnext"],
    // .js ファイルもTypeScriptで扱えるようにする
    "allowJs": true,
    // ライブラリの型チェックをスキップ (高速化)
    "skipLibCheck": true,
    // 厳格な型チェックを有効化 (推奨)
    "strict": true,
    // .ts → .js への変換ファイルは出力しない (Next.jsが代行)
    "noEmit": true,
    // CommonJS と ES Modules の混在を許可
    "esModuleInterop": true,
    // モジュールシステムは最新の ES Modules
    "module": "esnext",
    // モジュール解決方式 (バンドラー用)
    "moduleResolution": "bundler",
    // .json ファイルを import 可能にする
    "resolveJsonModule": true,
    // 各ファイルを独立したモジュールとして扱う
    "isolatedModules": true,
    // JSX をそのまま残す (Next.jsが変換)
    "jsx": "preserve",
    // 増分コンパイルを有効化 (再ビルドが速くなる)
    "incremental": true,
    "plugins": [
      // Next.js 用の TypeScript プラグイン
      { "name": "next" }
    ],
    // パスエイリアスの定義
    "paths": {
      // @/foo は src/foo を指す
      "@/*": [".src/*"]
    }
  },
  // 型チェック対象
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  // 型チェック対象から除外
  "exclude": ["node_modules"]
}
```

next.config.ts の例（最小構成）

```
// next.config.ts

// Next.js の設定型をインポート (type 付きで型のみ取得)
import type { NextConfig } from "next";

// 設定オブジェクトの型を NextConfig に固定
const nextConfig: NextConfig = {
  /* ここに追加の設定を書く (例: images.remotePatterns で外部画像ドメインを許可など) */
};

// デフォルトエクスポート (Next.js が自動で読み込む)
export default nextConfig;
```

2. Tailwind CSS の基礎

2.1 ユーティリティファーストとは

従来の CSS では、まずクラス名を考え、そのクラスに対してスタイルを定義していました。

```
/* 従来の CSS */
/* card という名前のクラスにスタイルを定義 */
.card {
  /* 背景色を白に */
  background-color: white;
  /* 角を 8px の半径で丸める */
  border-radius: 8px;
  /* 内側の余白を 16px に */
  padding: 16px;
  /* 薄い影を下方向に */
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}
```

```
<!-- HTML 側で class 属性に名前を指定して上記スタイルを適用 -->
<div class="card">...</div>
```

Tailwind CSS では、あらかじめ用意された小さなユーティリティクラスを組み合わせでスタイリングします。

```
<!-- Tailwind CSS: 1つ1つが小さな役割を持つクラスを並べる -->
<!-- bg-white: 背景白 / rounded-lg: 大きめの角丸 / p-4: 内側16px余白 / shadow-sm: 控えめな影 -->
<div class="bg-white rounded-lg p-4 shadow-sm">...</div>
```

それぞれのクラスが1つの CSS プロパティに対応しています。

クラス	対応する CSS
<code>bg-white</code>	<code>background-color: white;</code>
<code>rounded-lg</code>	<code>border-radius: 0.5rem;</code>
<code>p-4</code>	<code>padding: 1rem;</code>
<code>shadow-sm</code>	<code>box-shadow: 0 1px 2px rgba(0,0,0,0.05);</code>

ユーティリティファーストのメリット:

- CSS ファイルを作る必要がない (JSX だけで完結する)
- クラス名を考える時間がゼロになる
- デザインシステム (余白・色・サイズ) が統一される
- 使っていないスタイルが自動で削除される (バンドルサイズが小さい)

ユーティリティファーストのデメリット:

- JSX のクラス名が長くなることがある
- 最初はクラス名を覚えるのに時間がかかる

2.2 よく使うクラス一覧

本チュートリアルで頻繁に使うクラスを以下にまとめます。この表をブックマークしておくと、開発中に素早く参照できます。

余白 (Spacing)

クラス	CSS	説明
<code>p-4</code>	<code>padding: 1rem; (16px)</code>	内側の余白 (全方向)
<code>px-4</code>	<code>padding-left: 1rem; padding-right: 1rem;</code>	内側の余白 (左右)
<code>py-2</code>	<code>padding-top: 0.5rem; padding-bottom: 0.5rem;</code>	内側の余白 (上下)
<code>pt-4</code>	<code>padding-top: 1rem;</code>	内側の余白 (上のみ)
<code>pb-4</code>	<code>padding-bottom: 1rem;</code>	内側の余白 (下のみ)
<code>m-4</code>	<code>margin: 1rem;</code>	外側の余白 (全方向)
<code>mx-auto</code>	<code>margin-left: auto; margin-right: auto;</code>	要素を水平方向に中央揃え
<code>mt-8</code>	<code>margin-top: 2rem;</code>	外側の余白 (上のみ)
<code>mb-4</code>	<code>margin-bottom: 1rem;</code>	外側の余白 (下のみ)
<code>space-y-4</code>	子要素間に <code>margin-top: 1rem;</code>	子要素の縦方向の間隔
<code>gap-4</code>	<code>gap: 1rem;</code>	Flex/Grid の子要素間の間隔

数値の法則: Tailwind の数値は `0.25rem` (4px) 単位です。 `p-1` = 4px、 `p-2` = 8px、 `p-4` = 16px、 `p-8` = 32px のように、数値を4倍するとピクセル値になります。

色 (Colors)

クラス	説明
<code>text-gray-900</code>	濃いグレーの文字色（ほぼ黒）
<code>text-gray-600</code>	中間のグレーの文字色
<code>text-gray-400</code>	薄いグレーの文字色
<code>text-blue-600</code>	青い文字色（リンクなど）
<code>text-red-600</code>	赤い文字色（エラーメッセージなど）
<code>text-green-600</code>	緑の文字色（成功メッセージなど）
<code>text-white</code>	白い文字色
<code>bg-white</code>	白い背景色
<code>bg-gray-50</code>	ごく薄いグレーの背景色
<code>bg-gray-100</code>	薄いグレーの背景色
<code>bg-blue-600</code>	青い背景色（ボタンなど）
<code>bg-red-600</code>	赤い背景色（削除ボタンなど）
<code>border-gray-200</code>	薄いグレーのボーダー色
<code>border-gray-300</code>	ボーダー色（フォーム入力欄など）

色の濃さ: 50 が最も薄く、950 が最も濃い色です。100 刻みで用意されています。gray-50（ほぼ白）、gray-500（中間）、gray-900（ほぼ黒）のようなイメージです。

タイポグラフィ (Typography)

クラス	CSS	説明
<code>text-xs</code>	<code>font-size: 0.75rem;</code>	極小テキスト (12px)
<code>text-sm</code>	<code>font-size: 0.875rem;</code>	小さいテキスト (14px)
<code>text-base</code>	<code>font-size: 1rem;</code>	通常テキスト (16px)
<code>text-lg</code>	<code>font-size: 1.125rem;</code>	やや大きいテキスト (18px)
<code>text-xl</code>	<code>font-size: 1.25rem;</code>	大きいテキスト (20px)
<code>text-2xl</code>	<code>font-size: 1.5rem;</code>	見出し (24px)
<code>text-3xl</code>	<code>font-size: 1.875rem;</code>	大見出し (30px)
<code>font-bold</code>	<code>font-weight: 700;</code>	太字
<code>font-semibold</code>	<code>font-weight: 600;</code>	やや太字
<code>font-medium</code>	<code>font-weight: 500;</code>	やや太字 (控えめ)
<code>font-normal</code>	<code>font-weight: 400;</code>	通常の太さ
<code>text-center</code>	<code>text-align: center;</code>	テキスト中央揃え
<code>truncate</code>	<code>overflow: hidden; text-overflow: ellipsis; white-space: nowrap;</code>	テキストが長い場合に「...」で省略

レイアウト（Layout / Flexbox / Grid）

クラス	CSS	説明
<code>flex</code>	<code>display: flex;</code>	Flexbox コンテナにする
<code>flex-col</code>	<code>flex-direction: column;</code>	子要素を縦に並べる
<code>items-center</code>	<code>align-items: center;</code>	交差軸方向に中央揃え
<code>justify-between</code>	<code>justify-content: space-between;</code>	両端に配置
<code>justify-center</code>	<code>justify-content: center;</code>	主軸方向に中央揃え
<code>grid</code>	<code>display: grid;</code>	Grid コンテナにする
<code>grid-cols-1</code>	<code>grid-template-columns: repeat(1, 1fr);</code>	1列のグリッド
<code>grid-cols-2</code>	<code>grid-template-columns: repeat(2, 1fr);</code>	2列のグリッド
<code>grid-cols-3</code>	<code>grid-template-columns: repeat(3, 1fr);</code>	3列のグリッド
<code>w-full</code>	<code>width: 100%;</code>	横幅を親要素いっぱい
<code>max-w-7xl</code>	<code>max-width: 80rem;</code>	最大横幅（1280px）
<code>min-h-screen</code>	<code>min-height: 100vh;</code>	最小高さを画面いっぱい
<code>hidden</code>	<code>display: none;</code>	要素を非表示にする

ボーダー・角丸・影（Borders / Radius / Shadow）

クラス	CSS	説明
<code>border</code>	<code>border-width: 1px;</code>	1px のボーダーを追加
<code>border-2</code>	<code>border-width: 2px;</code>	2px のボーダーを追加
<code>rounded</code>	<code>border-radius: 0.25rem;</code>	少し角丸
<code>rounded-md</code>	<code>border-radius: 0.375rem;</code>	中程度の角丸
<code>rounded-lg</code>	<code>border-radius: 0.5rem;</code>	大きめの角丸
<code>rounded-full</code>	<code>border-radius: 9999px;</code>	完全な丸（バッジやアイコンに）
<code>shadow-sm</code>	小さなシャドウ	控えめな影
<code>shadow</code>	中程度のシャドウ	標準的な影
<code>shadow-lg</code>	大きなシャドウ	目立つ影

状態変化 (Hover / Focus)

クラス	説明
<code>hover:bg-blue-700</code>	マウスを載せたときに背景色を濃い青に変更
<code>hover:text-blue-600</code>	マウスを載せたときに文字色を青に変更
<code>hover:shadow-md</code>	マウスを載せたときに影を追加
<code>focus:outline-none</code>	フォーカス時のデフォルトのアウトラインを消す
<code>focus:ring-2</code>	フォーカス時に2pxのリングを表示
<code>focus:ring-blue-500</code>	フォーカスリングの色を青に設定
<code>transition</code>	CSS トランジションを有効にする (スムーズなアニメーション)
<code>duration-200</code>	トランジション時間を200msに設定
<code>cursor-pointer</code>	マウスカーソルをポインター (指) にする
<code>disabled:opacity-50</code>	無効状態のときに半透明にする
<code>disabled:cursor-not-allowed</code>	無効状態のときにカーソルを「禁止」にする

2.3 レスポンシブデザイン

Tailwind CSS では、ブレイクポイントのプレフィックスを付けることで、画面サイズに応じたスタイルを簡単に適用できます。

プレフィックス	最小幅	対象デバイス
(なし)	0px	すべて (モバイルファースト)
<code>sm:</code>	640px	大きめスマートフォン
<code>md:</code>	768px	タブレット
<code>lg:</code>	1024px	ノートPC
<code>xl:</code>	1280px	デスクトップ
<code>2xl:</code>	1536px	大画面

重要: Tailwind はモバイルファースト設計です。プレフィックスなしのクラスがモバイル (最小画面) のスタイルになり、プレフィックス付きのクラスが「その画面サイズ以上」に適用されます。

実例: レスポンシブなグリッドレイアウト

```
{/* grid: グリッド表示 / grid-cols-1: 1列 / md:grid-cols-2: 768px以上で2列 / lg:grid-cols-3: 1024px以上で3列 / gap-4: 子要素間に16px間隔 */}
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {/* 1つ目のセル */}
  <div>カード1</div>
  {/* 2つ目のセル */}
  <div>カード2</div>
  {/* 3つ目のセル */}
  <div>カード3</div>
</div>
```

この例では次のように表示が変わります。

画面サイズ	適用されるクラス	表示
0px ~ 767px	<code>grid-cols-1</code>	1列表示（カードが縦に並ぶ）
768px ~ 1023px	<code>md:grid-cols-2</code>	2列表示
1024px ~	<code>lg:grid-cols-3</code>	3列表示

2.4 「このクラスを付けたらこう表示される」具体例

実際にボタンを例にして、クラスを1つずつ追加していくとどう変わるか見てみましょう。

Step 1: テキストだけ（クラスなし）

```
{/* 何もクラスを付けないボタン。ブラウザのデフォルト見た目になる */}
<button>書籍を登録する</button>
```

ブラウザデフォルトのボタンが表示されます。装飾はほぼなく、小さな灰色の枠線と白い背景のボタンです。

Step 2: 背景色と文字色を追加

```
{/* bg-blue-600: 青い背景 / text-white: 白い文字 */}
<button className="bg-blue-600 text-white">書籍を登録する</button>
```

青い背景に白い文字のボタンになります。ただし、余白がなくテキストがギリギリです。

Step 3: 余白を追加

```
{/* px-4: 左右の内側余白16px / py-2: 上下の内側余白8px */}  
<button className="bg-blue-600 text-white px-4 py-2">書籍を登録する</button>
```

左右に16px、上下に8pxの余白が入り、見た目が改善されます。

Step 4: 角丸を追加

```
{/* rounded-md: 中程度の角丸 (border-radius: 6px) */}  
<button className="bg-blue-600 text-white px-4 py-2 rounded-md">書籍を登録する</button>
```

角が丸くなり、モダンな印象になります。

Step 5: ホバーエフェクトとトランジションを追加

```
{/* hover:bg-blue-700: マウスホバー時に少し濃い青へ */}  
{/* transition: 変化をスムーズに / duration-200: 200ミリ秒かけて変化 */}  
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700  
transition duration-200">  
  書籍を登録する  
</button>
```

マウスを載せると背景色が少し濃くなり、200ms かけてスムーズに変化します。

Step 6: フォーカスリングを追加

```
{/* focus:outline-none: デフォルトの黒い枠を消す */}  
{/* focus:ring-2: フォーカス時に2pxのリング */}  
{/* focus:ring-blue-500: リング色は青 */}  
{/* focus:ring-offset-2: リングとボタンの間に2pxの隙間 */}  
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700  
transition duration-200 focus:outline-none focus:ring-2 focus:ring-blue-500 focus:ring-  
offset-2">  
  書籍を登録する  
</button>
```

Tab キーでフォーカスしたときに、ボタンの周りに青いリングが表示されます。これはアクセシビリティ（キーボード操作対応）のために重要です。

完成形: 本チュートリアルで使うボタンスタイル

```
{/* font-medium: やや太字 (500) */}  
{/* disabled:opacity-50: disabled属性付きのときは半透明 */}  
{/* disabled:cursor-not-allowed: disabled時はカーソルを「禁止」マークに */}  
<button className="bg-blue-600 text-white px-4 py-2 rounded-md font-medium hover:bg-blue-700 transition duration-200 focus:outline-none focus:ring-2 focus:ring-blue-500 focus:ring-offset-2 disabled:opacity-50 disabled:cursor-not-allowed">  
  書籍を登録する  
</button>
```

3. 必要なパッケージのインストール

プロジェクトのディレクトリに移動してから、必要なパッケージをインストールします。

```
# プロジェクトフォルダに移動  
cd book-management  
  
# npm install: package.json に依存パッケージを追加し、node_modules/ にダウンロード  
# @supabase/supabase-js: Supabase 公式の JavaScript/TypeScript クライアントライブラリ  
npm install @supabase/supabase-js
```

3.1 各パッケージの役割

パッケージ名	役割	説明
<code>@supabase/supabase-js</code>	Supabase クライアント	Supabase（データベース、認証、ストレージ）と通信するための公式 JavaScript/TypeScript ライブラリです。このパッケージを使って、ブラウザから直接 Supabase の PostgreSQL データベースにデータの読み書きができます。
<code>next</code> （プリインストール済み）	Next.js フレームワーク	React ベースのフルスタックフレームワークです。ルーティング、サーバーサイドレンダリング、API ルートなどの機能を提供します。
<code>react</code> （プリインストール済み）	React ライブラリ	UI コンポーネントを構築するためのライブラリです。 <code>useState</code> や <code>useEffect</code> などのフック（Hook）を使って、動的な UI を作成します。
<code>react-dom</code> （プリインストール済み）	React DOM	React コンポーネントをブラウザの DOM（画面）にレンダリングするためのライブラリです。
<code>typescript</code> （プリインストール済み）	TypeScript	JavaScript に型システムを追加した言語です。開発時の型チェックに使われます。
<code>tailwindcss</code> （プリインストール済み）	Tailwind CSS	ユーティリティファーストの CSS フレームワークです。 <code>create-next-app</code> で Tailwind CSS を選択した場合、自動的にインストールされます。
<code>eslint</code> （プリインストール済み）	ESLint	コードの品質チェックツールです。バグの原因になりやすいパターンを検出します。

`@supabase/supabase-js` だけインストールすれば十分な理由: 本チュートリアル の書籍管理アプリでは、Supabase との通信がメインの外部依存です。UI は Tailwind CSS（プリインストール済み）で構築し、フォーム処理は React の標準機能で実現します。追加の UI ライブラリやフォームライブラリは使用しないため、シンプルな構成を保てます。

3.2 パッケージがインストールされたことの確認

`package.json` を開き、`dependencies` に `@supabase/supabase-js` が追加されていることを確認します。

```
{
  "dependencies": {
    // 今インストールしたSupabaseクライアント (^は後述)
    "@supabase/supabase-js": "^2.x.x",
    // Next.js本体
    "next": "15.x.x",
    // React本体
    "react": "^19.x.x",
    // React DOM
    "react-dom": "^19.x.x"
  }
}
```

^ の意味: **^2.49.1** は「メジャーバージョン 2 の範囲で最新版を許容する」という意味です。**2.49.1** から **2.99.99** まではインストール可能ですが、**3.0.0** はインストールされません。これにより、破壊的変更を避けつつ最新のバグ修正やパフォーマンス改善を受けられます。

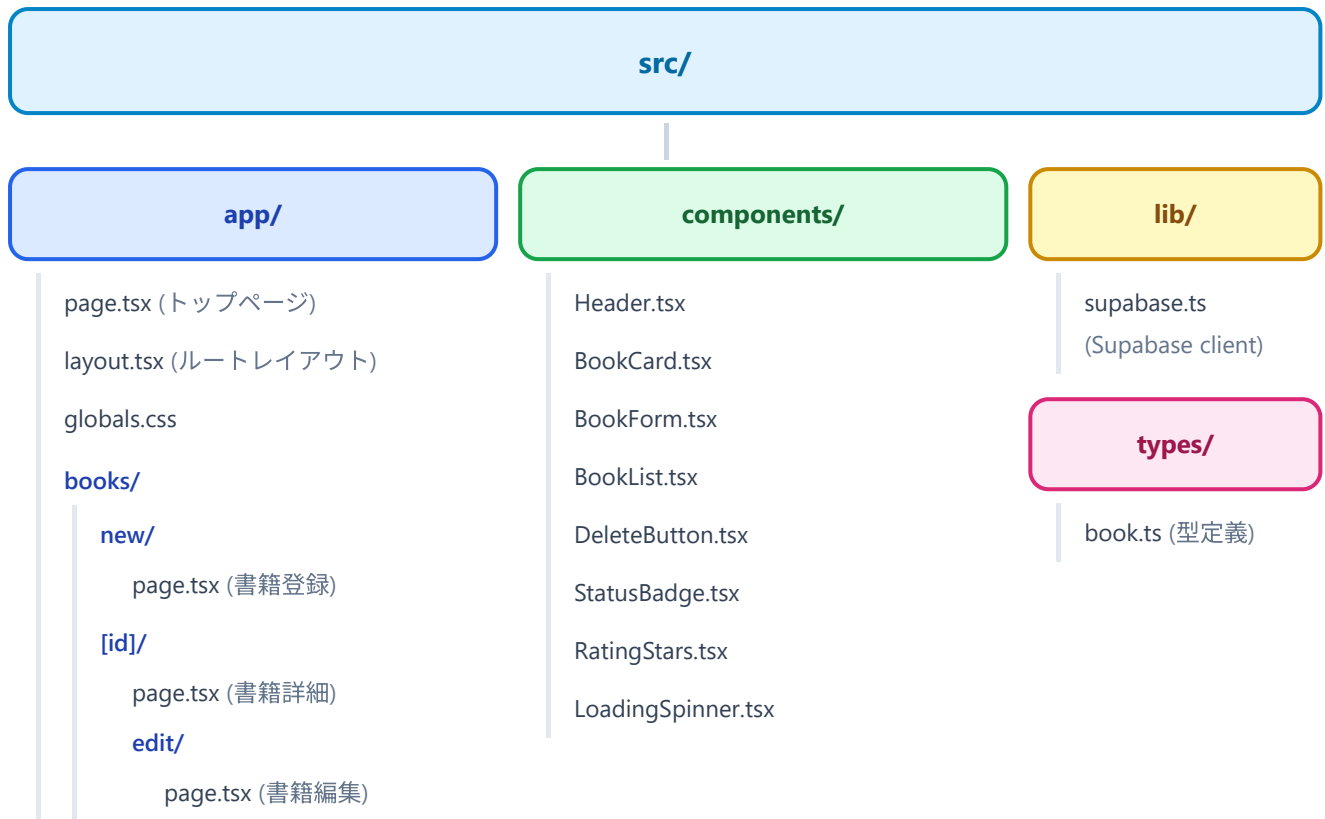
セマンティックバージョンing (SemVer) の補足: バージョン番号は **メジャー.マイナー.パッチ** の形 (例: **2.49.1**) になっています。 - **メジャー (2)** : 互換性のない変更 (古い書き方が動かなくなる可能性あり) - **マイナー (49)** : 機能追加 (後方互換あり) - **パッチ (1)** : バグ修正

^ は「マイナーとパッチの自動アップデートはOK、メジャーは固定」という意味のルールです。

4. プロジェクト構成の作成

4.1 ディレクトリ構成

書籍管理アプリでは、以下のディレクトリ構成を採用します。



テキスト形式でも示します。


```

src/
├─ app/
│   ├─ page.tsx
│   ├─ layout.tsx
│   ├─ globals.css
│   └─ books/
│       ├─ new/
│       │   └─ page.tsx
│       └─ [id]/
│           ├─ page.tsx
│           └─ edit/
│               └─ page.tsx
├─ components/
│   ├─ Header.tsx
│   ├─ BookCard.tsx
│   ├─ BookForm.tsx
│   ├─ BookList.tsx
│   ├─ DeleteButton.tsx
│   ├─ StatusBadge.tsx
│   ├─ RatingStars.tsx
│   └─ LoadingSpinner.tsx
├─ lib/
│   └─ supabase.ts
└─ types/
    └─ book.ts

```

App Router のルート。フォルダ名 = URL になる
 # / にアクセスしたときのページ (トップ: 書籍一覧)
 # 全ページ共通の枠 (HTMLタグ、ヘッダーなど)
 # アプリ全体のグローバルCSS (Tailwindディレクティブ含む)
 # /books 配下のページをまとめるフォルダ
 # /books/new ページ用のフォルダ
 # 書籍登録ページ
 # [id] は動的セグメント。/books/任意のID にマッチ
 # 書籍詳細ページ (/books/123 など)
 # /books/[id]/edit ページ用のフォルダ
 # 書籍編集ページ
 # 再利用可能なUI部品を置くフォルダ
 # 共通ヘッダー (全ページ上部のナビ)
 # 書籍カード (一覧の各アイテム)
 # 書籍登録・編集フォーム
 # 書籍一覧表示 (BookCardを並べる)
 # 削除ボタン (確認ダイアログ付き)
 # ステータスバッジ (読書中など)
 # 評価 (星) 表示
 # ローディング表示 (くるくる回るやつ)
 # アプリ全体で使うユーティリティを置くフォルダ
 # Supabase クライアントの初期化・エクスポート
 # TypeScript 型定義を置くフォルダ
 # Book / BookInsert / BookUpdate などの型

4.2 各ファイルの役割

ページファイル（ `app/` ディレクトリ）

ファイル	URL パス	役割
<code>app/page.tsx</code>	<code>/</code>	トップページ。データベースから書籍の一覧を取得し、BookList コンポーネントで表示します。
<code>app/layout.tsx</code>	全ページ共通	ルートレイアウト。 <code><html></code> と <code><body></code> タグ、ヘッダーなどの全ページ共通要素を定義します。
<code>app/globals.css</code>	全ページ共通	Tailwind CSS のディレクティブとカスタムスタイルを定義します。
<code>app/books/new/page.tsx</code>	<code>/books/new</code>	書籍登録ページ。BookForm コンポーネントを使い、新しい書籍の情報を入力・保存します。
<code>app/books/[id]/page.tsx</code>	<code>/books/123</code>	書籍詳細ページ。URL の <code>[id]</code> 部分に書籍の ID が入り、その書籍の詳細情報を表示します。
<code>app/books/[id]/edit/page.tsx</code>	<code>/books/123/edit</code>	書籍編集ページ。既存の書籍情報を読み込み、BookForm コンポーネントで編集・更新します。

[id] とは？（動的ルーティング） フォルダ名を角括弧 `[]` で囲むと「動的ルート」になります。`/books/1`、`/books/2`、`/books/abc` のように、`[id]` の部分に任意の値が入ります。ページコンポーネントの `params` プロパティからその値を取得できます。

コンポーネントファイル（ **components/** ディレクトリ）

ファイル	役割
Header.tsx	アプリ上部に表示されるナビゲーションバー。アプリ名と「書籍を登録する」リンクを含みます。全ページで共通して表示されます。
BookCard.tsx	書籍一覧の中の1冊分の表示カード。タイトル、著者、ステータスバッジ、評価（星）を表示します。クリックすると詳細ページに遷移します。
BookForm.tsx	書籍の登録・編集に使うフォーム。タイトル、著者、ステータス、評価、メモの入力欄を持ちます。新規登録と編集の両方で共通して使用します。
BookList.tsx	BookCard を並べて一覧表示するコンポーネント。レスポンシブなグリッドレイアウトで配置します。書籍が0件の場合は「書籍が登録されていません」というメッセージを表示します。
DeleteButton.tsx	書籍を削除するボタン。クリック時に確認ダイアログを表示し、承諾した場合のみ削除を実行します。
StatusBadge.tsx	読書ステータス（「未読」「読書中」「読了」）を色付きバッジで表示するコンポーネント。ステータスに応じて色が変わります。
RatingStars.tsx	5段階の星評価を表示するコンポーネント。黄色い星と灰色の星で評価を視覚化します。
LoadingSpinner.tsx	データ読み込み中に表示するスピナー（回転アニメーション）。API からのレスポンスを待っている間、ユーザーに読み込み中であることを伝えます。

ユーティリティファイル（ **lib/** ディレクトリ）

ファイル	役割
supabase.ts	Supabase クライアントの初期化と設定。環境変数から Supabase の URL と API キーを読み取り、アプリ全体で使えるクライアントインスタンスをエクスポートします。

型定義ファイル（ **types/** ディレクトリ）

ファイル	役割
book.ts	書籍データの TypeScript 型定義。 Book 型（データベースから取得する完全なデータ）、 BookInsert 型（新規登録時に送るデータ）、 BookUpdate 型（更新時に送るデータ）を定義します。

4.3 ディレクトリとファイルの作成

以下のコマンドで必要なディレクトリとファイルを一括作成します。

▼このコードがやること（先に日本語で）：これから書いていくコードを置くための「フォルダ（ディレクトリ）」をまとめて作ります。`mkdir` はフォルダを作るコマンドで、`-p` を付けると途中の親フォルダもまとめて作ってくれます。`[id]` のように角括弧が入った特殊なフォルダ名はそのままとシェルに誤解されるため、`\` で記号を打ち消して（エスケープして）います。うまくいかなければ、コメントにある通り VS Code から手作業で作ってもOKです。

```
# ディレクトリの作成
# mkdir: Make Directory の略。フォルダを作成するコマンド
# -p: 親フォルダが存在しなければ一緒に作る（既に存在してもエラーにならない）
mkdir -p src/app/books/new
# [id] は角括弧の意味を持つ特殊文字なので、シェルで誤解されないようにバックスラッシュでエスケープ
mkdir -p src/app/books/[id\]/edit
mkdir -p src/components
mkdir -p src/lib
mkdir -p src/types
```

注意: `[id]` は角括弧を含むため、シェルによってはエスケープが必要です。うまくいかない場合は、VS Code のエクスプローラーからフォルダを手動で作成してください。

▼このコードがやること（先に日本語で）：さきほど作ったフォルダの中に、これから編集していく「空っぽのファイル」を一気に用意します。`touch` は中身が空のファイルを作るコマンドで、中身は後の節で書き込んでいきます。Windows の PowerShell には `touch` がないので、その場合はコメントにある `New-Item -ItemType File フォイル名` で代用してください。

```
# ファイルの作成（中身は後ほど記述）
# touch: 空のファイルを作成するコマンド（既に存在する場合は最終更新時刻だけ変える）
# Windows の PowerShell では touch がないので、代わりに「New-Item -ItemType File ファイル名」を使う
# 共通ヘッダー
touch src/components/Header.tsx
# 書籍カード
touch src/components/BookCard.tsx
# 書籍フォーム
touch src/components/BookForm.tsx
# 書籍一覧
touch src/components/BookList.tsx
# 削除ボタン
touch src/components/DeleteButton.tsx
# ステータスバッジ
touch src/components/StatusBadge.tsx
# 評価表示
touch src/components/RatingStars.tsx
# ローディング
touch src/components/LoadingSpinner.tsx
# Supabase クライアント
touch src/lib/supabase.ts
# Book 型定義
touch src/types/book.ts
```

5. Supabase クライアントの設定

5.1 環境変数の取得

Supabase クライアントの設定には、以下の2つの情報が必要です。

1. Supabase URL -- あなたのプロジェクトの API エンドポイント
2. Supabase Anon Key -- 匿名アクセス用の API キー

これらは Supabase のダッシュボードから取得できます。

取得手順:

1. [Supabase のダッシュボード](#) にアクセスします
2. 書籍管理アプリ用のプロジェクトを選択します
3. 左のサイドバーから「Project Settings」（歯車アイコン）をクリックします
4. 「API」タブをクリックします

5. 以下の情報をコピーします: - **Project URL**: `https://xxxxxxxxxxxxxxxxx.supabase.co` の形式 - **Project API keys** の anon / public キー: `eyJhbGciOi...` で始まる長い文字列

5.2 .env.local の作成

プロジェクトのルートディレクトリ（ `book-management/` の直下）に `.env.local` ファイルを作成します。

▼ このコードがやること（先に日本語で）：Supabase（データベース）に接続するための「URL」と「鍵（キー）」を、コードの外側の設定ファイルに書き出します。こうした外部から渡す設定値を「環境変数」と呼び、**キー=値** の形で1行ずつ書きます。**NEXT_PUBLIC_** を頭に付けた変数だけがブラウザ側からも読めるようになる点がポイントです。

xxxxxxx の部分は、必ず自分の Supabase プロジェクトの値に置き換えてください。

```
# book-management/.env.local
# このファイルは ENV ファイルと呼ばれ、「キー=値」の形で環境変数を1行ずつ書く
# NEXT_PUBLIC_ プレフィックスを付けると Next.js がブラウザ側にも公開してくれる

# Supabase プロジェクトのURL（ダッシュボードからコピー）
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxxxxxxxx.supabase.co

# Supabase の Anon Key（匿名アクセス用の公開鍵。長いJWTトークン形式）
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwiaWF0IjE1NjQzMDE2MDAwfQ.xxxxxxxxxxxxxxxxxxxxxxx
XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

NEXT_PUBLIC_ プレフィックスの重要性: Next.js では、環境変数に `NEXT_PUBLIC_` プレフィックスを付けると、ブラウザ側の JavaScript からアクセスできるようになります。プレフィックスがない環境変数はサーバー側でのみ使用可能です。

Supabase の Anon Key はブラウザから直接 API を呼び出すために使うため、`NEXT_PUBLIC_` が必要です。Anon Key は Row Level Security (RLS) で保護されているため、公開しても安全です（前の章で RLS を設定しました）。

ENV ファイルの種類と読み込み順 (Next.js) : Next.js では複数の `.env` 系ファイルを使い分けられます。同じ変数が複数のファイルにある場合、上のものが優先されます。

ファイル	いつ読み込まれるか	Gitに入れる？
<code>.env.local</code>	全環境（開発・本番ともに）。ただし <code>next test</code> は除く	入れない（.gitignoreで除外）
<code>.env.development</code>	開発時（ <code>npm run dev</code> ）のみ	入れてOK（秘密情報を含まないなら）
<code>.env.production</code>	本番時（ <code>npm run start</code> ）のみ	入れてOK（秘密情報を含まないなら）
<code>.env</code>	すべての環境（デフォルト値）	入れてOK（秘密情報を含まないなら）

読み込み優先順位（高 → 低）：`.env.local` > `.env.development` / `.env.production` > `.env`

覚え方:「`.local` が付くファイルは個人専用 = Gitに入れない = 秘密情報を入れてOK」。`.env.local` だけは絶対にコミットしない、と覚えれば事故を防げます。

`.env.local` がGitにアップロードされない理由: `create-next-app` が自動生成した `.gitignore` ファイルには `.env*.local` が含まれています。そのため、`.env.local` は Git にコミットされず、API キーが公開リポジトリに漏洩する心配はありません。

必ず `xxxxxxxxxxxxxxxxxx` の部分を、自分のプロジェクトの値に置き換えてください。

5.3 lib/supabase.ts の作成

`src/lib/supabase.ts` ファイルに以下のコードを記述します。

▼ このコードがやること（先に日本語で）：データベースと話すための「窓口（Supabase クライアント）」を1か所で作り、アプリ全体で使い回せるようにします。さきほど `.env.local` に書いた URL とキーを読み込み、それを使って接続用のオブジェクトを組み立てて `export`（外に公開）します。もし設定が抜けていたら、原因がすぐ分かるエラーをわざと出すようにしているのが安心ポイントです。1行ずつの意味はコード内のコメントと、下の解説表で確認できます。

```

// src/lib/supabase.ts
// このファイルは、アプリ全体で共有する Supabase クライアントを作成・エクスポートする役割

// @supabase/supabase-js パッケージから createClient 関数を「名前付きインポート」する
// {} で囲むのが名前付きインポートの構文。export されている特定の関数だけを取り出す
import { createClient } from "@supabase/supabase-js";

// 環境変数から Supabase の接続情報を取得
// process.env: Node.js 由来のグローバルオブジェクト。.env.local の中身がここに入る
// NEXT_PUBLIC_ で始まる変数は、ビルド時にブラウザ側のコードにも埋め込まれる
// SupabaseのプロジェクトURL
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
// Supabase の Anon Key
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

// 環境変数が設定されていない場合にわかりやすいエラーメッセージを表示
// !supabaseUrl は「supabaseUrl が undefined や空文字なら true」になる条件
if (!supabaseUrl) {
  // throw: エラーを「投げる」。アプリの実行を停止し、コンソールにエラーを出力する
  // new Error(...): JavaScript標準のエラーオブジェクトを作る
  throw new Error(
    "環境変数 NEXT_PUBLIC_SUPABASE_URL が設定されていません。" +
    ".env.local ファイルを確認してください。"
  );
}

// AnonKey についても同じく未設定チェックを行う
if (!supabaseAnonKey) {
  throw new Error(
    "環境変数 NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
    ".env.local ファイルを確認してください。"
  );
}

// Supabase クライアントを作成してエクスポート
// アプリ全体で同じインスタンスを使い回す（シングルトンパターン）
// export const: 他のファイルから import できる変数を宣言
// createClient(URL, Key): 第1引数 URL、第2引数 認証キーで Supabase に接続するクライアントを作成
export const supabase = createClient(supabaseUrl, supabaseAnonKey);

```

コードの解説:

行	解説
<pre>import { createClient } from "@supabase/supabase-js";</pre>	Supabase の公式ライブラリから <code>createClient</code> 関数をインポートします。この関数が Supabase との接続を確立します。中括弧 <code>{}</code> で囲むのが「named import（名前付きインポート）」で、ライブラリが <code>export { createClient }</code> のように公開している特定の関数だけを取り出します。
<pre>process.env.NEXT_PUBLIC_SUPABASE_URL</pre>	<code>.env.local</code> に定義した環境変数を読み取ります。<code>process.env</code> は Node.js が環境変数にアクセスするためのオブジェクトです。
<pre>if (!supabaseUrl)</pre>	環境変数が未設定（<code>undefined</code>）の場合にエラーを投げます。<code>.env.local</code> の作成忘れや typo を早期に検出できます。
<pre>createClient(supabaseUrl, supabaseAnonKey)</pre>	Supabase クライアントを生成します。第1引数が API の URL、第2引数が認証キーです。
<pre>export const supabase</pre>	生成したクライアントを <code>supabase</code> という名前でエクスポートします。他のファイルから <code>import { supabase } from "@/lib/supabase";</code> でインポートして使います。

import 文の種類（重要）：JavaScript/TypeScript には3種類のインポート方法があります。- **default import**: `import React from "react";` のように中括弧なし。各ファイルにつき1つだけ **export default** できる主要なものを取り出す。- **named import**: `import { useState } from "react";` のように中括弧あり。複数の名前を指定して取り出せる（例: `import { useState, useEffect } from "react";`）。- **type import**: `import type { Book } from "@/types/book";` のように **type** キーワード付き。型のみをインポートし、ビルド時に取り除かれる（JavaScriptには型がないため）。

シングルトンパターンとは？ アプリ全体で「ただ1つのインスタンス」を共有する設計のこと。`supabase.ts` で1回だけ `createClient` を呼び、できた `supabase` オブジェクトを全ファイルで使い回します。毎回新しく作るとリソースの無駄、設定がバラバラになる、などの問題が起きるので、共通化しておくのが定石です。

Server Component と Client Component で Supabase を使うときの違い（重要）：

Next.js の App Router では、コンポーネントが「Server Component」（サーバー側で実行）か「Client Component」（ブラウザ側で実行）かを区別します。それぞれで Supabase を使う際の注意点が異なります。

観点	Server Component	Client Component
ファイル先頭の宣言	（宣言なし。デフォルト）	<code>"use client"</code> を書く
実行場所	サーバー（あなたのPC or 本番サーバー）	ユーザーのブラウザ
使える React フック	使えない（ <code>useState</code> など不可）	使える
データ取得	<code>await supabase.from(...)</code> を直接書ける	<code>useEffect</code> 内で呼ぶ必要あり
環境変数	サーバー専用変数（ <code>SUPABASE_SERVICE_ROLE_KEY</code> 等）も読める	<code>NEXT_PUBLIC_</code> 付きの変数のみ
認証情報	サーバー側でCookieからセッションを取る場合がある	ブラウザのlocalStorage等にセッション保存
このチュートリアル	主に静的な表示で使用	フォーム入力や削除などインタラクションで使用

本チュートリアルでは、書籍一覧表示は Client Component（フォーム入力との連携のため `"use client"`）で実装します。より高度な用途では、Server Component から直接Supabaseを呼ぶこともできます。

▼ コードを1つずつ分解して解説

上のコードを、初心者がつまづきやすい部分ごとに1つずついねいに見ていきましょう。

解説1: createClient 関数を「名前付きインポート」する

```
import { createClient } from "@supabase/supabase-js";
```

- `import ... from "..."` は、別のファイルやライブラリから機能を取り込むための構文です。
- `{ createClient }` のように波括弧 `{ }` で囲むのが「名前付きインポート」で、そのライブラリが公開している特定の名前の機能だけをピンポイントで取り出します。
- ここでは `@supabase/supabase-js`（インストールした Supabase 公式ライブラリ）から、`createClient`（接続用の窓口を作る関数）だけを取り出しています。
- この関数を後で呼ぶことで、アプリと Supabase をつなぐ「クライアント」が作られます。

用語:「ライブラリ」とは、誰かが作って公開してくれた便利な機能のかたまりのことです。`npm install` で自分のプロジェクトに取り込み、`import` で必要な部分だけ呼び出して使います。

解説2: 環境変数から接続情報を読み取る

```
// SupabaseのプロジェクトURL
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
// Supabase の Anon Key
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

- `process.env`（プロセス・エンブ）は、環境変数の入れ物です。さきほど `.env.local` に書いた値が、ここに入ってきます。
- `process.env.NEXT_PUBLIC_SUPABASE_URL` と書くと、`.env.local` の `NEXT_PUBLIC_SUPABASE_URL=...` の値（URL）を取り出せます。
- 取り出した値を `supabaseUrl` と `supabaseAnonKey` という名前の定数（`const`）に入れて、あとで使いやすくしています。
- URL とキーをコードに直接書かず、外部の設定ファイル経由で渡すことで、**秘密情報の漏洩を防ぎ**、環境ごとに値を切り替えられます。

用語:「環境変数」とは、コードの外側からプログラムへ渡す設定値のことです。API キーのような秘密情報や、開発用と本番用で変えたい値を、コード本体と分けて管理するために使います。

解説3: 設定が抜けていたらわざとエラーを出す

```
if (!supabaseUrl) {  
  throw new Error(  
    "環境変数 NEXT_PUBLIC_SUPABASE_URL が設定されていません。" +  
    ".env.local ファイルを確認してください。"  
  );  
}
```

- `!supabaseUrl` の `!` (エクスクラメーション) は「否定」の記号で、「`supabaseUrl` が空っぽ (`undefined` や空文字) なら `true`」という意味になります。
- つまりこの `if` は「URL がちゃんと取れなかったとき」だけ中に入ります。
- `throw new Error(...)` は、エラーをわざと発生させて処理を止める書き方です。原因 (環境変数が未設定) と対処法 (`.env.local` を確認) をメッセージに書いておくと、後で自分が困りません。
- `+` は文字列をつなげる演算子で、長いメッセージを2行に分けて書いています。Anon Key 側も同じチェックをしています。

用語: 「例外 (throw / Error)」とは、処理を続けられない問題が起きたときに発生させる中断の合図です。早い段階でわかりやすく失敗させることを「フェイルファスト (fail fast)」と呼び、原因究明を楽にします。

解説4: クライアントを作って export する

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

- `createClient(URL, キー)` を呼ぶと、Supabase と通信するための窓口オブジェクトが1つ作られます。第1引数が接続先 URL、第2引数が認証キーです。
- それを `supabase` という定数に入れ、先頭に `export` を付けることで、他のファイルからこの `supabase` を `import` して使えるようにしています。
- このファイルで1回だけ作り、アプリ全体で同じものを使い回します (毎回作り直さない)。
- 他のファイルでは `import { supabase } from "@/lib/supabase";` と書けば、この窓口を呼び出せます。

用語: 「export」は、ファイルの中で作ったものを外部に公開する指定です。逆に他ファイルでそれを受け取るのが「import」です。この2つでファイル間の機能の受け渡しを行います。

6. 型定義ファイルの作成

6.1 types/book.ts の作成

`src/types/book.ts` ファイルに以下のコードを記述します。この型定義は、アプリ全体で書籍データの構造を統一するために使います。

▼ このコードがやること（先に日本語で）：「書籍データはどんな形をしているか」を TypeScript の型として書き出し、アプリ全体で構造を統一します。型とは「このデータには title と author が必須」といった設計図のことで、間違っただけのデータを使うとコードを動かす前に気づけます。場面ごとに必要な項目が異なるため、取得用の **Book** ・登録用の **BookInsert** ・更新用の **BookUpdate** の3つに分けている点が要チェックです。後半ではステータスの日本語ラベルや色も定数としてまとめています。

```
// src/types/book.ts
// 書籍データに関する型と定数をまとめて定義するファイル

/**
 * 読書ステータスの型定義
 *
 * - "unread" : 未読 (まだ読んでいない)
 * - "reading" : 読書中 (今読んでいる途中)
 * - "finished": 読了 (読み終わった)
 */
// type: TypeScript の型エイリアス (型に名前を付ける構文)
// "unread" | "reading" | "finished": ユニオン型。この3つの文字列リテラルのいずれかしか取れない
// export: 他のファイルからこの型をインポートできるようにする
export type BookStatus = "unread" | "reading" | "finished";

/**
 * 書籍データの型定義 (データベースから取得した完全なデータ)
 *
 * Supabase の books テーブルの行データに対応します。
 * すべてのフィールドが含まれており、データベースから取得した
 * 書籍データはこの型に一致します。
 */
// type Book = { ... }: オブジェクトの形を表す型を Book という名前で定義
export type Book = {
  /** 書籍ID (UUID形式、Supabase が自動生成) */
  // UUID は "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" 形式の文字列
  id: string;

  /** 書籍のタイトル (必須) */
  // 例: "リーダブルコード"
  title: string;

  /** 著者名 (必須) */
  // 例: "Dustin Boswell"
  author: string;

  /** 読書ステータス (デフォルト: "unread") */
  // 上で定義した BookStatus 型 (3つの文字列のどれか)
  status: BookStatus;

  /** 評価 (1~5の整数、未評価の場合は null) */
  // number | null: 数値か null のどちらか (ユニオン型)
  rating: number | null;

  /** メモ・感想 (任意、未入力の場合は null) */
  // 文字列または null
  memo: string | null;

  /** 作成日時 (ISO 8601 形式の文字列、Supabase が自動生成) */

```

```

// 例: "2024-01-15T10:30:00.000Z"
created_at: string;

/** 更新日時 (ISO 8601 形式の文字列、Supabase が自動生成) */
// 行が更新されるたびに Supabase のトリガーが書き換える
updated_at: string;
};

/**
 * 書籍の新規登録時に送るデータの型定義
 *
 * id, created_at, updated_at はデータベースが自動生成するため、
 * 送信データには含めません。
 * status, rating, memo はオプション (省略可能) です。
 */
export type BookInsert = {
  /** 書籍のタイトル (必須) */
  // 必須項目 (? が付いていない)
  title: string;

  /** 著者名 (必須) */
  // 必須項目
  author: string;

  /** 読書ステータス (省略時は "unread" がデフォルト) */
  // ?: オプション。省略可能。省略時の値はDB側のデフォルトで補う
  status?: BookStatus;

  /** 評価 (1~5、省略時は null) */
  // 省略してもよいし、明示的に null を送ってもよい
  rating?: number | null;

  /** メモ・感想 (省略時は null) */
  // 同上
  memo?: string | null;
};

/**
 * 書籍の更新時に送るデータの型定義
 *
 * 更新したいフィールドだけを送ればよいため、
 * すべてのフィールドがオプションです。
 * Partial<BookInsert> と同じ意味ですが、明示的に定義しています。
 */
export type BookUpdate = {
  /** 書籍のタイトル */
  // すべて ? 付き (更新時は変更したいフィールドのみ送る)
  title?: string;

  /** 著者名 */

```

```

// 同上
author?: string;

/** 読書ステータス */
// 同上
status?: BookStatus;

/** 評価 (1~5、null で評価をクリア) */
// null を明示的に送ると評価をリセットできる
rating?: number | null;

/** メモ・感想 (null でメモをクリア) */
// 同上
memo?: string | null;
};

/**
 * ステータスの表示ラベル定義
 *
 * コンポーネントでステータスを日本語表示するときに使います。
 * 例: statusLabels["reading"] → "読書中"
 */
// Record<BookStatus, string>: TypeScript の組み込み型。
// 「キーが BookStatus 型、値が string 型のオブジェクト」を意味する
// 3つのキー (unread, reading, finished) すべてが必須になる
export const statusLabels: Record<BookStatus, string> = {
  // "unread" の表示用ラベル
  unread: "未読",
  // "reading" の表示用ラベル
  reading: "読書中",
  // "finished" の表示用ラベル
  finished: "読了",
};

/**
 * ステータスに対応する色のクラス定義
 *
 * StatusBadge コンポーネントでバッジの色を切り替えるときに使います。
 * Tailwind CSS のクラス名を直接指定しています。
 */
// Record<BookStatus, { bg: string; text: string }>:
// キーが BookStatus、値が { bg: string; text: string } 型のオブジェクト
// 各ステータスごとに「背景色クラス」と「文字色クラス」を持つ
export const statusColors: Record<
  BookStatus,
  { bg: string; text: string }
> = {
  // 未読: グレー系 (控えめ)
  unread: {
    // 背景は薄いグレー

```



```

    bg: "bg-gray-100",
    // 文字は濃いめのグレー
    text: "text-gray-700",
  },
  // 読書中: 青系 (進行中の印象)
  reading: {
    // 背景は薄い青
    bg: "bg-blue-100",
    // 文字は濃いめの青
    text: "text-blue-700",
  },
  // 読了: 緑系 (完了の印象)
  finished: {
    // 背景は薄い緑
    bg: "bg-green-100",
    // 文字は濃いめの緑
    text: "text-green-700",
  },
};

```

コードの解説:

型 / 定数	用途
<code>BookStatus</code>	読書ステータスを3つのリテラル型のユニオンで定義します。 <code>"unread" "reading" "finished"</code> 以外の値を代入しようとすると、TypeScript がコンパイルエラーを出します。
<code>Book</code>	データベースの <code>books</code> テーブルの1行に対応する型です。 <code>supabase.from("books").select("*")</code> の返り値はこの型の配列になります。
<code>BookInsert</code>	新規登録時に必要なデータの型です。 <code>title</code> と <code>author</code> が必須、それ以外はオプションル（ <code>?</code> 付き）です。 <code>id</code> や <code>created_at</code> はデータベースが自動で付与するため含みません。
<code>BookUpdate</code>	更新時に送るデータの型です。すべてのフィールドがオプションルで、変更したい項目だけを送ります。
<code>statusLabels</code>	ステータスの文字列を日本語ラベルに変換するためのマッピングオブジェクトです。 <code>Record<BookStatus, string></code> は「キーが <code>BookStatus</code> 型、値が <code>string</code> 型のオブジェクト」を意味します。
<code>statusColors</code>	ステータスに対応する Tailwind CSS のクラス名をマッピングしたオブジェクトです。StatusBadge コンポーネントでバッジの背景色と文字色を動的に切り替えるために使います。

Supabase の自動生成型（Database 型）について: 大規模プロジェクトでは Supabase CLI で `npx supabase gen types typescript` を実行し、データベースの定義から TypeScript の型を自動生成することができます。生成された型は次のような形をしています。

```
typescript export type Database = { public: { Tables: { books: { Row: { id: string; title: string; ... }; // SELECTで返ってくる行の型 Insert: { title: string; author: string; ... }; // INSERT時に渡す型 Update: { title?: string; ... }; // UPDATE時に渡す型 }; }; }; };
```

本チュートリアルでは学習のしやすさのため、手書きで `Book` , `BookInsert` , `BookUpdate` を定義しています。仕組みは同じです。

なぜ3つの型を分けるのか？

データベースとやり取りする場面によって、必要なフィールドが異なります。

- 取得（SELECT）：すべてのフィールドが返ってくる → `Book` 型
- 登録（INSERT）：`id` や `created_at` は不要 → `BookInsert` 型
- 更新（UPDATE）：変更したいフィールドだけ送る → `BookUpdate` 型

型を分けることで、各場面で正確なデータ構造を強制でき、バグを防げます。

▼ コードを1つずつ分解して解説

上の型定義を、初心者がつまずきやすい部分ごとに1つずついていねいに見ていきましょう。

解説1: 決まった3つの文字列しか入らない「ユニオン型」

```
export type BookStatus = "unread" | "reading" | "finished";
```

- `type 名前 = ...` は、よく使うデータの形に自分で名前を付ける構文（型エイリアス）です。
- `"unread" | "reading" | "finished"` の `|`（縦棒）は「または」を意味し、この3つの文字列のどれかしかが入れられない、という制約になります。
- たとえば `"yomu"` のような違う文字列を入れようとすると、コードを動かす前に TypeScript がエラーで教えてくれます。タイプミスを防げるのが利点です。
- 先頭の `export` で、他のファイルからこの `BookStatus` を使えるようにしています。

用語:「ユニオン型」とは、複数の候補のうちのいずれかを表す型です。決まった選択肢しか取らない値（ステータスや種別など）を安全に扱うのに向いています。

解説2: 取得用の完全なデータ型 `Book`

```
export type Book = {
  id: string;
  title: string;
  author: string;
  status: BookStatus;
  rating: number | null;
  memo: string | null;
  created_at: string;
  updated_at: string;
};
```

- `type Book = { ... }` は、オブジェクト（複数の項目を持つデータ）の形を定義しています。データベースの `books` テーブルの1行ぶんに対応します。
- 各行は `項目名: 型` の形で、「`title` は文字列」「`rating` は数値」のように、項目ごとに入る値の種類を決めています。
- `status: BookStatus` のように、解説1で作った型を部品として再利用できます。
- `number | null` は「数値、または null（値なし）」という意味で、未評価のときに `null` を許すための書き方です。

用語: 「null」とは「値が存在しない」ことを表す特別な値です。 `number | null` のように書くと、「数値が入るか、まだ何も入っていないか」の両方を表現できます。

解説3: 場面で型を分ける（Insert と Update）

```
export type BookInsert = {
  title: string;
  author: string;
  status?: BookStatus;
  rating?: number | null;
  memo?: string | null;
};
```

- `BookInsert` は新規登録のときに送るデータの型です。 `id` や `created_at` はデータベースが自動で付けるので、ここには含めません。
- 項目名の後ろの `?`（クエスチョン）は「省略してもよい（オプション）」という印です。 `status?` は、書かなくてもエラーにならず、DB 側のデフォルト値が使われます。
- 一方 `BookUpdate` は更新用で、全項目に `?` が付いています。変更したい項目だけを送れば済むようにするためです。
- このように場面ごとに必要な項目を分けることで、「登録時に `id` を送ってしまう」といった間違いを型で防げます。

用語:「オプション（ ? ）」とは、そのプロパティがあってもなくてもよいという指定です。必須項目には付けず、省略を許したい項目にだけ付けます。

解説4: ステータスごとの値をまとめる **Record** 型

```
export const statusLabels: Record<BookStatus, string> = {
  unread: "未読",
  reading: "読書中",
  finished: "読了",
};
```

- これは型ではなく**実際の値（定数）**です。**"reading"** のような内部用の文字列を、**"読書中"** という画面表示用の日本語に変換するための対応表です。
- **Record<BookStatus, string>** は「キーが **BookStatus**、値が文字列のオブジェクト」という型で、3つのステータス全部を漏れなく書くことを強制してくれます。
- 使うときは **statusLabels["reading"]** のように書くと **"読書中"** が取り出せます。
- 同じ仕組みの **statusColors** では、値が文字列ではなく **{ bg, text }**（背景色クラスと文字色クラス）のオブジェクトになっています。

用語:「Record」は TypeScript の組み込み型で、キーの型 **K** と値の型 **V** を指定して**対応表（マップ）の形**を作ります。選択肢ごとの設定値をまとめるのに便利です。

7. ルートレイアウトの設定

7.1 app/layout.tsx の作成

src/app/layout.tsx はすべてのページを囲む「外枠」のようなものです。HTML の **<html>** と **<body>** タグを含み、全ページ共通のヘッダーや CSS をここで設定します。

以下のコードで **src/app/layout.tsx** を書き換えます（**create-next-app** が生成したデフォルトの内容を完全に置き換えます）。

▼ このコードがやること（先に日本語で）：すべてのページを包む「共通の外枠」を作ります。**<html>** や **<body>** タグ、日本語フォント、ブラウザのタブに出るタイトル、そして全ページ上部のヘッダーをここでまとめて設定します。各ページの中身は **{children}**（チルドレン：差し込み口）の場所にはめ込まれる、という仕組みがポイントです。フォントやレイアウトの細かい指定はコメントと解説表で1つずつ確認できます。

```

// src/app/layout.tsx
// 全ページに適用される「ルートレイアウト」。<html> と <body> はここでしか書かない

// type import: Metadata 型だけをインポート（実体はビルド後のコードに残らない）
import type { Metadata } from "next";

// named import: next/font/google から Noto_Sans_JP 関数だけを取り出す
// next/font は Next.js が用意した、フォント最適化のための機能
import { Noto_Sans_JP } from "next/font/google";

// CSS のサイドエフェクトインポート（左辺なし）
// このファイルを読み込んだだけで CSS が適用される
import "./globals.css";

// default import: Header コンポーネントを取り込む
// @/components/Header は src/components/Header（パスエイリアス @/ = src/）
import Header from "@/components/Header";

/**
 * フォント設定
 *
 * Google Fonts から Noto Sans JP（日本語対応フォント）を読み込みます。
 * next/font/google を使うことで、フォントファイルがビルド時に
 * 自動的に最適化・セルフホスティングされます。
 * （外部の Google Fonts サーバーにリクエストが飛ばないため高速）
 */
const notoSansJP = Noto_Sans_JP({
  // 読み込むサブセット（文字種）。日本語はデフォルトで含まれる
  subsets: ["latin"],
  // 使用するフォントの太さ（通常 / やや太字 / 太字）
  weight: ["400", "500", "700"],
  // フォント読み込み中は代替フォントで表示し、完了後に切り替える
  display: "swap",
  // 重要なフォントなので、HTMLの <link rel="preload"> で先読みする
  preload: true,
});

/**
 * メタデータ設定
 *
 * ブラウザのタブに表示されるタイトルや、
 * 検索エンジンに表示される説明文を設定します。
 */
// export const metadata: Next.js が自動でこの変数を読み取って <head> に反映する
// 型は Metadata（先ほど type import したもの）
export const metadata: Metadata = {
  // <title> タグの中身（ブラウザのタブ表示）
  title: "書籍管理アプリ",
  // <meta name="description"> の中身（検索結果に出る説明文）

```

```

description:
  "読んだ本、読んでいる本、これから読む本を管理するWebアプリケーション",
};

/**
 * ルートレイアウトコンポーネント
 *
 * すべてのページはこのレイアウトの中にレンダリングされます。
 * { children } には各ページのコンテンツが入ります。
 *
 * 構造:
 * <html>
 *   <body>
 *     <Header />          ← 全ページ共通のヘッダー
 *     <main>{children}</main> ← 各ページの内容
 *   </body>
 * </html>
 */
// export default function: ファイルの主要な関数として1つだけ default エクスポート
// RootLayout: コンポーネントの名前。Next.js が自動でこの関数を呼ぶ
// { children }: 分割代入で props から children を取り出している
// Readonly<{ children: React.ReactNode }>: 引数の型。children は読み取り専用、React の任意のノード
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  // JSX を return する。1つの親要素 (<html>) で全体を包む必要がある
  return (
    // lang="ja": HTML の言語属性を日本語に。スクリーンリーダーや検索エンジン用
    <html lang="ja">
      {/* body の className: notoSansJP のクラス名 + Tailwind の背景色と最小高さ */}
      {/* テンプレートリテラル `${...}` で文字列を結合 */}
      <body className={`${notoSansJP.className} bg-gray-50 min-h-screen`} >
        {/* 共通ヘッダーを最上部に配置 */}
        <Header />
        {/* main: ページのメインコンテンツ領域 */}
        {/* max-w-7xl: 最大幅1280px / mx-auto: 左右マージン自動で中央寄せ */}
        {/* px-4 sm:px-6 lg:px-8: 画面サイズに応じて左右パディングを変える (モバイル16px → タブレット24px → PC32px) */}
        {/* py-8: 上下パディング32px */}
        <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
          {/* children: 各ページの中身がここに差し込まれる */}
          {children}
        </main>
      </body>
    </html>
  );
}

```

```
};
}
```

コードの解説:

部分	解説
<code>import type { Metadata } from "next";</code>	Next.js のメタデータ型をインポートします。 <code>type</code> キーワードは「型のみインポート」を意味し、ランタイムには影響しません（ビルド時に消える）。
<code>import { Noto_Sans_JP } from "next/font/google";</code>	Google Fonts から Noto Sans JP フォントをインポートします。 <code>next/font</code> を使うと、フォントがビルド時にダウンロードされ、パフォーマンスが最適化されます。
<code>import "./globals.css";</code>	Tailwind CSS のグローバルスタイルを読み込みます。この1行がないと Tailwind のクラスが機能しません。 <code>./</code> は同じフォルダにあるファイルを指す相対パス。
<code>import Header from "@components/Header";</code>	共通ヘッダーコンポーネントをインポートします。 <code>@/</code> は <code>src/</code> ディレクトリのエイリアスです。これにより <code>../.../components/Header</code> のように相対パスを書かなくて済みます。
<code>subsets: ["latin"]</code>	ラテン文字のサブセットを読み込みます。日本語文字はデフォルトで含まれます。
<code>weight: ["400", "500", "700"]</code>	使用するフォントウェイト（太さ）を指定します。400 = 通常、500 = やや太字、700 = 太字。指定しない太さは使えなくなる代わりに、ダウンロードサイズが減ります。
<code>display: "swap"</code>	フォントの読み込み方を指定します。 <code>"swap"</code> はフォント読み込み完了前にシステムフォントで表示し、読み込み後に切り替えます（FOIT: Flash Of Invisible Text を防ぐ）。
<code>export const metadata</code>	ページのメタデータを定義します。 <code>title</code> がブラウザのタブに表示され、 <code>description</code> が検索エンジンの説明文に使われます。Next.js がこの export を自動的に検出して <code><head></code> タグに反映します。
<code>lang="ja"</code>	HTML の言語を日本語に設定します。スクリーンリーダーや検索エンジンが正しく言語を認識できます。
<code>bg-gray-50</code>	<code><body></code> の背景色をごく薄いグレーにします。真っ白よりも目に優しく、カードなどの白い要素が際立ちます。
<code>min-h-screen</code>	<code><body></code> の最小高さを画面いっぱいにします。コンテンツが少ないページでも背景色が画面全体に適用されます。
<code>max-w-7xl mx-auto</code>	<code><main></code> の最大幅を 1280px に制限し、左右の margin を auto にして中央寄せにします。大画面でもコンテンツが横に広がりすぎるのを防ぎます。
<code>px-4 sm:px-6 lg:px-8</code>	画面サイズに応じて左右の padding を変えます。モバイルでは 16px、タブレットでは 24px、PCでは 32px です。
<code>{children}</code>	各ページのコンテンツがここに挿入されます。 <code>/</code> にアクセスすれば <code>app/page.tsx</code> の内容が、 <code>/books/new</code> にアクセスすれば <code>app/books/new/page.tsx</code> の内容が入ります。

▼ コードを1つずつ分解して解説

上のレイアウトを、初心者がつまづきやすい部分ごとに1つずついねいに見ていきましょう。

解説1: 3種類のインポートを使い分ける

```
import type { Metadata } from "next";
import { Noto_Sans_JP } from "next/font/google";
import "./globals.css";
import Header from "@/components/Header";
```

- 1行目の `import type` は「型だけを取り込む」インポートで、ビルド後のコードには残りません（型は実行時には不要なため）。
- 2行目の `{ Noto_Sans_JP }` は波括弧付きの「名前付きインポート」で、日本語フォントを読み込む関数を取り出しています。
- 3行目の `import "./globals.css";` は左側に名前がない特殊な形で、「このファイルを読み込むだけ」を意味します。これで Tailwind のスタイルが全ページに効きます。
- 4行目の `import Header` は波括弧なしの「デフォルトインポート」。@/ は `src/` の近道（パスエイリアス）なので、`@/components/Header` は `src/components/Header` を指します。

用語:「パスエイリアス（@/）」とは、長い相対パス（`../../components/...`）を短く書くための別名です。`src/` を起点にできるので、ファイルを移動してもインポート文を直さずに済みます。

解説2: フォントを最適化して読み込む

```
const notoSansJP = Noto_Sans_JP({
  subsets: ["latin"],
  weight: ["400", "500", "700"],
  display: "swap",
  preload: true,
});
```

- `Noto_Sans_JP({ ... })` のように、フォント関数に**設定オブジェクト**を渡して呼び出します。戻り値を `notoSansJP` に保存し、あとで `<body>` に適用します。
- `weight: ["400", "500", "700"]` は使う太さ（通常・やや太字・太字）の指定で、使う分だけ読み込むことでファイルサイズを抑えます。
- `display: "swap"` は、フォント読み込み中はいったん代替フォントで表示し、読み込み完了後に切り替える設定です。文字が一瞬消える現象を防ぎます。
- `next/font` を使うとフォントがビルド時にダウンロードされて自前配信されるため、表示が速く安定します。

用語:「フォントの最適化」とは、必要な文字種・太さだけをまとめてダウンロードし、表示の遅延やレイアウトのガタつきを減らす仕組みのことです。

解説3: メタデータでタブのタイトルなどを設定

```
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description:
    "読んだ本、読んでいる本、これから読む本を管理するWebアプリケーション",
};
```

- `metadata` という名前で `export` した値は、Next.js が自動で見つけて HTML の `<head>` に反映してくれます（自分で `<title>` を書く必要がありません）。
- `title` はブラウザのタブに表示される文字、`description` は検索結果に出る説明文になります。
- `: Metadata` は「この値は `Metadata` という型に従う」という指定で、解説1で `import type` したものです。項目名を間違えると型チェックで気づけます。

用語:「メタデータ」とは、ページ自体の内容ではなく、ページに付随する情報（タイトル・説明・OGP など）のことです。検索エンジンや SNS が参照します。

解説4: children に各ページが差し込まれる仕組み

```
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="ja">
      <body className={` ${notoSansJP.className} bg-gray-50 min-h-screen`} >
        <Header />
        <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
          {children}
        </main>
      </body>
    </html>
  );
}
```

- `RootLayout` は全ページを包む外枠コンポーネントです。引数の `{ children }` には、表示中の各ページの中身が自動的に入ってきます。
- `Readonly<{ children: React.ReactNode }>` は引数の型で、`React.ReactNode` は「React が表示できるあらゆる中身（要素・文字列など）」を表します。
- `className={ ${ notoSansJP.className } ... }` の ``${...}`` はテンプレートリテラルで、フォントのクラス名と Tailwind のクラスをつなげて1つの文字列にしています。
- `<Header />` を常に上に置き、その下の `<main>` 内の `{children}` 部分だけがページごとに入れ替わります。これで全ページ共通の枠を1か所で管理できます。

用語: 「children（チルドレン）」とは、コンポーネントの中に差し込まれる中身を受け取る特別な props です。レイアウトのように「枠だけ用意して中身は後から入れる」設計でよく使います。

7.2 globals.css の確認

`src/app/globals.css` は、`create-next-app` がデフォルトで生成した内容をそのまま使用します。以下の Tailwind CSS ディレクティブが含まれていることを確認してください（不要なデフォルトスタイルは削除して構いません）。

```
/* src/app/globals.css */

/* @tailwind: Tailwind CSS 専用のディレクティブ。PostCSS が処理時に大量のクラスに展開する */
/* リセットCSSと要素のベーススタイル */
@tailwind base;
/* component レイヤー（カスタムコンポーネントクラス） */
@tailwind components;
/* 全ユーティリティクラス（最も多くのコードが展開される） */
@tailwind utilities;
```

Tailwind CSS ディレクティブの意味:

ディレクティブ	役割
<code>@tailwind base;</code>	ブラウザ間の表示差異をなくすリセット CSS と、基本的な要素のスタイルを注入します。
<code>@tailwind components;</code>	コンポーネントクラス（ <code>container</code> など）を注入します。
<code>@tailwind utilities;</code>	ユーティリティクラス（ <code>flex</code> , <code>pt-4</code> , <code>text-center</code> など）をすべて注入します。未使用のクラスはビルド時に自動削除されます。

Next.js のバージョンによる違い: Next.js 15 以降のバージョンでは、`globals.css` の記法が `@import "tailwindcss";` のように変わっている場合があります。`create-next-app` が生成した内容がどちらの形式であっても、Tailwind CSS は正しく動作しますのでそのまま使用してください。

8. 共通ヘッダーコンポーネント

8.1 components/Header.tsx の作成

`src/components/Header.tsx` ファイルに以下のコードを記述します。

▼ このコードがやること（先に日本語で）: 全ページの上部に出る「ナビゲーションバー（共通ヘッダー）」を作ります。アプリ名のロゴと、「書籍一覧」「書籍を登録する」へのリンクを並べ、いま開いているページのリンクだけ色を変えて目立たせます。`usePathname()` という機能で現在の URL を取得するため、先頭に `"use client"` と書いてブラウザ側で動くコンポーネントにしている点がポイントです。スタイル（色や余白）の意味は各行のコメントを参照してください。

```
// src/components/Header.tsx
// 全ページ上部に表示する共通ヘッダー（ナビゲーションバー）

// "use client": このファイルを Client Component として宣言する特別な文字列
// 必ずファイルの一番上（importよりも前）に書く
// これがあるとブラウザで実行され、React フックや状態管理が使える
"use client";

// default import: Next.js の Link コンポーネント
// HTML の <a> よりも高速にページ遷移できる（SPA的な切り替え）
import Link from "next/link";

// named import: usePathname フック（現在のURLパスを取得）
// 注意: import元は "next/navigation"（App Router用）。Pages Router の "next/router" とは別物
import { usePathname } from "next/navigation";

/**
 * 共通ヘッダーコンポーネント
 *
 * すべてのページの上部に表示されるナビゲーションバーです。
 * - アプリ名（クリックでトップページに遷移）
 * - 「書籍を登録する」ボタン
 * を含みます。
 *
 * "use client" を使用する理由:
 * usePathname() フックを使用するため、Client Component にする必要があります。
 * Server Component では React のフック（useState, useEffect, usePathname 等）は使えません。
 */
// export default: このファイルを import するときの主要な対象
// function Header(): 関数コンポーネントの宣言。引数なし、JSX を返す
export default function Header() {
  // 現在のURLパスを取得（アクティブなナビゲーションリンクのスタイル変更に使用）
  // usePathname() の戻り値は "/" や "/books/new" のような文字列
  const pathname = usePathname();

  // JSX を return（ヘッダー全体の構造）
  return (
    // <header>: HTML5 のセマンティックタグ。ページ上部のヘッダーを示す
    // bg-white: 白い背景 / shadow-sm: 控えめな影 / border-b: 下方向に1pxのボーダー /
    border-gray-200: ボーダー色
    <header className="bg-white shadow-sm border-b border-gray-200">
      {/* 内側の div: ヘッダーの中身を中央寄せ&最大幅制限する */}
      {/* max-w-7xl: 最大幅1280px / mx-auto: 左右マージンautoで中央寄せ */}
      {/* px-4 sm:px-6 lg:px-8: 画面サイズに応じて左右パディング */}
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        {/* flex 行: ロゴと右側ナビを横並びに、両端配置、高さ64px */}
        {/* flex: フレックスボックス / items-center: 垂直中央揃え */}
        {/* justify-between: 左右に均等配置（ロゴが左、ナビが右） / h-16: 高さ64px */}

```

```

<div className="flex items-center justify-between h-16">
  {/* アプリ名 (ロゴ) */}
  {/* Link: ページリロードなしで遷移する Next.js のコンポーネント */}
  {/* href: 遷移先のパス */}
  <Link
    href="/"
    // text-xl: フォントサイズ20px / font-bold: 太字 / text-gray-900: 濃いグレー
    // hover:text-blue-600: ホバー時に青 / transition duration-200: 200msで滑らか
    に変化
    className="text-xl font-bold text-gray-900 hover:text-blue-600 transition
duration-200"
    >
    書籍管理アプリ
  </Link>

  {/* ナビゲーション */}
  {/* nav: HTML5 のナビゲーション用セマンティックタグ */}
  {/* space-x-4: 子要素間に16pxの水平方向マージン */}
  <nav className="flex items-center space-x-4">
    {/* 書籍一覧リンク */}
    <Link
      href="/"
      // テンプレートリテラル ${} で動的にクラスを切り替える
      // pathname === "/" が true なら青系 (現在地ハイライト)、false ならグレー系
      className={`px-3 py-2 rounded-md text-sm font-medium transition duration-
200 ${
        pathname === "/"
          // 現在地: 薄青背景 + 青文字
          ? "bg-blue-50 text-blue-700"
          // 通常: グレー文字、ホバーで反応
          : "text-gray-600 hover:text-gray-900 hover:bg-gray-50"
        }`}
      >
      書籍一覧
    </Link>

    {/* 書籍登録ボタン */}
    <Link
      href="/books/new"
      // px-4: 左右パディング16px / py-2: 上下パディング8px / rounded-md: 角丸
      // text-sm: 文字サイズ14px / font-medium: やや太字 / transition: 滑らか変化
      className={`px-4 py-2 rounded-md text-sm font-medium transition duration-
200 ${
        pathname === "/books/new"
          // 現在地: 濃い青背景 + 白文字
          ? "bg-blue-700 text-white"
          // 通常: 青背景、ホバーで濃く
          : "bg-blue-600 text-white hover:bg-blue-700"
        }`}
      >

```

```
        書籍を登録する
      </Link>
    </nav>
  </div>
</div>
</header>
);
}
```

コードの解説:

部分	解説
<code>"use client";</code>	このファイルを Client Component として宣言します。Next.js の App Router では、デフォルトですべてのコンポーネントが Server Component（サーバーで実行）です。しかし <code>usePathname()</code> などの React フックを使うには Client Component にする必要があります。ファイルの先頭にこの宣言を書くだけで切り替えられます。
<code>import Link from "next/link";</code>	Next.js のリンクコンポーネントです。HTML の <code><a></code> タグの代わりに使います。ページ遷移時にページ全体をリロードせず、必要な部分だけを更新するため高速です。
<code>import { usePathname } from "next/navigation";</code>	現在のURLパスを取得するフックです。例えば <code>/books/new</code> にいるときは <code>"/books/new"</code> が返ります。
<code>pathname === "/"</code>	現在のページがトップページかどうかを判定し、条件に応じてスタイルを切り替えます。現在いるページのリンクには背景色を付けてハイライト表示します。
<code>bg-white shadow-sm border-b border-gray- 200</code>	ヘッダーの背景を白にし、下に薄いシャドウとボーダーを付けてコンテンツ部分との区切りを明確にします。
<code>max-w-7xl mx- auto</code>	ヘッダー内のコンテンツの最大幅を 1280px にし、中央寄せにします。 <code>layout.tsx</code> の <code><main></code> と同じ幅にすることで整った見た目になります。
<code>flex items- center justify- between h-16</code>	Flexbox で子要素を横並びにし、垂直方向に中央揃え、左右に均等配置、高さ64pxを設定します。
<code>space-x-4</code>	ナビゲーションリンク間に 16px の水平方向の間隔を設けます。

▼ コードを1つずつ分解して解説

上のヘッダーを、初心者がつまづきやすい部分ごとに1つずついねいに見ていきましょう。

解説1: なぜ先頭に "use client" が必要か

```
"use client";
```

- この1行は、ファイルを「Client Component（ブラウザ側で動く部品）」として宣言する特別な合図です。必ず import より前、ファイルの一番上に入ります。
- Next.js の App Router では、何も書かないとコンポーネントはサーバー側で動く設定になります。サーバー側では `useState` などの React の機能や、URL を取る `usePathname()` が使えません。
- このヘッダーは現在地にに応じて見た目を変えるため `usePathname()` を使います。そのために `"use client"` を付けてブラウザ側で動くようにしています。

用語: 「Client Component」とは、ユーザーのブラウザ上で実行されるコンポーネントです。クリックや入力などの操作、React フックを使いたい部品はこちらにします。

解説2: Link と usePathname を取り込む

```
import Link from "next/link";  
import { usePathname } from "next/navigation";
```

- `Link` は、ページをまるごと再読み込みせずに素早く切り替えるための Next.js 専用のリンク部品です。HTML の `<a>` の代わりに使います。
- `usePathname` は「いま開いている URL のパス」（例: `"/"` や `"/books/new"`）を取得するフックです。
- import 元に注意が必要で、`usePathname` は App Router 用の `"next/navigation"` から取り込みます（旧方式の `"next/router"` とは別物です）。

用語: 「フック（Hook）」とは、`use` で始まる React の特別な関数です。状態や現在の URL など、コンポーネントに機能を「引っかけて」追加するために使います。

解説3: 現在のパスを取得する

```
const pathname = usePathname();
```

- `usePathname()` を呼ぶと、今表示しているページの URL パスが文字列で返ってきます。それを `pathname` に保存します。
- この値を後で `pathname === "/"` のように比較して、「いま見ているページのリンクだけ色を変える」ハイライト表示に使用します。
- ページを移動すると `pathname` の値も自動で変わり、ヘッダーの見た目も更新されます。

用語:「パス (pathname)」とは、URL のうちドメイン名より後ろの部分 (`/books/new` など) のことです。どのページにいるかを判別する手がかりになります。

解説4: 現在地によってクラスを切り替える

```
<Link
  href="/"
  className={`px-3 py-2 rounded-md text-sm font-medium transition duration-200 ${
    pathname === "/"
      ? "bg-blue-50 text-blue-700"
      : "text-gray-600 hover:text-gray-900 hover:bg-gray-50"
  }}
>
  書籍一覧
</Link>
```

- `className={ ...${ ... } }` のテンプレートリテラルで、共通のクラスと条件で変わるクラスを1つの文字列に合体させています。
- `条件 ? Aの値 : Bの値` は三項演算子（条件分岐の短い書き方）で、「`pathname === "/"` が真なら A、偽なら B」を選びます。
- いまトップページにいれば薄い青背景 + 青文字 (`bg-blue-50 text-blue-700`) でハイライトし、そうでなければグレー文字でホバー時に反応するスタイルになります。
- `href="/"` で遷移先を指定し、クリックすると `Link` が高速にトップページへ切り替えます。

用語:「三項演算子 (`条件 ? A : B`)」とは、`if/else` を1行で書ける短縮形です。値を2択で切り替えたいとき、JSX の中などで手軽に使えます。

9. 開発サーバーの起動と動作確認

9.1 仮のトップページを作成

動作確認のため、`src/app/page.tsx` を以下の内容に書き換えます（`create-next-app` のデフォルト内容を置き換えます）。

▼ このコードがやること（先に日本語で）：トップページ（URL が `/` のページ）に、動作確認用の仮の画面を表示します。見出しと説明文、白いカード、3色のステータスバッジを並べて、Tailwind CSS のスタイルがちゃんと効いているかを目で確かめられるようにしています。中身はまだ仮で、本物の書籍一覧は次の章で作る、という位置づけです。クラス名（`bg-white` など）の意味は各行のコメントにあります。

```
// src/app/page.tsx
// URL "/" にアクセスしたときに表示されるトップページ

// export default function: ファイルのメインコンポーネントとしてエクスポート
// HomePage: コンポーネント名（任意の名前でOK。Next.jsはdefault exportを自動的にページとして扱う）
export default function HomePage() {
  // JSX を return（このページの中身）
  return (
    // 全体を div で囲む。複数の要素を return するには親要素が必要
    <div>
      {/* h1: 大見出し / text-3xl: 30px / font-bold: 太字 / text-gray-900: 濃いグレー / mb-4: 下マージン16px */}
      <h1 className="text-3xl font-bold text-gray-900 mb-4">
        書籍一覧
      </h1>
      {/* p: 段落 / text-gray-600: 中間グレー / mb-8: 下マージン32px */}
      <p className="text-gray-600 mb-8">
        登録された書籍がここに表示されます。
      </p>

      {/* 動作確認用: Tailwind CSS のスタイルが正しく適用されるかチェック */}
      {/* bg-white: 白背景 / rounded-lg: 角丸大 / shadow-sm: 控えめな影 */}
      {/* border border-gray-200: 1pxの薄いグレーボーダー / p-6: 内側パディング24px */}
      <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
        {/* h2: 中見出し / text-lg: 18px / font-semibold: やや太字 (600) */}
        <h2 className="text-lg font-semibold text-gray-800 mb-2">
          セットアップ完了
        </h2>
        {/* text-sm: 14px の小さめテキスト */}
        <p className="text-gray-600 text-sm">
          Next.js + Tailwind CSS + Supabase の開発環境が正しくセットアップされました。
          次の章で、このページに実際の書籍一覧を表示します。
        </p>
        {/* バッジを横並びにする領域 */}
        {/* mt-4: 上マージン16px / flex: 横並び / space-x-2: 子要素間に8px間隔 */}
        <div className="mt-4 flex space-x-2">
          {/* 未読バッジ */}
          {/* inline-block: インラインブロック表示 / bg-gray-100: 薄グレー背景 / text-gray-700: 濃グレー文字 */}
          {/* text-xs: 12px / px-2.5 py-1: パディング / rounded-full: 完全な丸 (カプセル形) / font-medium: 中太字 */}
          <span className="inline-block bg-gray-100 text-gray-700 text-xs px-2.5 py-1 rounded-full font-medium">
            未読
          </span>
          {/* 読書中バッジ: 青系 */}
          <span className="inline-block bg-blue-100 text-blue-700 text-xs px-2.5 py-1 rounded-full font-medium">

```

```

        読書中
      </span>
      { /* 読了バッジ: 緑系 */ }
      <span className="inline-block bg-green-100 text-green-700 text-xs px-2.5 py-1
rounded-full font-medium">
        読了
      </span>
    </div>
  </div>
</div>
);
}

```

▼ コードを1つずつ分解して解説

上の仮トップページを、初心者がつまずきやすい部分ごとに1つずつていねいに見ていきましょう。

解説1: ページは default export の関数で作る

```

export default function HomePage() {
  return (
    <div>
      ...
    </div>
  );
}

```

- `app/page.tsx` の中で `export default` した関数が、その URL（ここでは `/`）に表示されるページの本体になります。Next.js が自動でこの関数を探して呼び出します。
- 関数名（`HomePage`）は自由に決められます。重要なのは名前ではなく「`export default` であること」です。
- `return ( ... )` の中に画面の中身（JSX）を書きます。複数の要素を返すときは、必ず1つの親要素（ここでは外側の `<div>`）で包む必要があります。

用語: 「default export」とは、1ファイルにつき1つだけ指定できる「主役の書き出し」です。Next.js のページやコンポーネントは、これでファイルの代表を決めます。

解説2: Tailwind のクラスで見出しと説明を装飾

```
<h1 className="text-3xl font-bold text-gray-900 mb-4">
  書籍一覧
</h1>
<p className="text-gray-600 mb-8">
  登録された書籍がここに表示されます。
</p>
```

- JSX では HTML の `class` 属性のかわりに `className` を使います（`class` は JavaScript の予約語のため）。
- `className` に Tailwind のクラスをスペース区切りで並べると、それぞれが1つのスタイルになります。`text-3xl`（文字サイズ30px）、`font-bold`（太字）、`text-gray-900`（濃いグレー）など。
- `mb-4` / `mb-8` は下方向の余白（margin-bottom）で、数字が大きいくほど余白も広がります（4=16px、8=32px）。

用語: 「className」とは、JSX で CSS クラスを指定するための属性名です。HTML の `class` と同じ役割ですが、React では `className` と書く決まりになっています。

解説3: カードとステータスバッジを並べる

```
<div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
  <h2 className="text-lg font-semibold text-gray-800 mb-2">
    セットアップ完了
  </h2>
  ...
  <div className="mt-4 flex space-x-2">
    <span className="inline-block bg-gray-100 text-gray-700 text-xs px-2.5 py-1
rounded-full font-medium">
      未読
    </span>
    ...
  </div>
</div>
```

- 外側の `<div>` が白いカードです。`bg-white`（白背景）、`rounded-lg`（角丸）、`shadow-sm`（薄い影）、`border`（枠線）、`p-6`（内側の余白24px）を組み合わせで作っています。
- バッジを囲む `<div>` の `flex space-x-2` で、3つのバッジを横並びにし、間に8pxの隙間を空けています。
- 各バッジ（`<span>`）の `rounded-full` でカプセル型の丸みを付け、`bg-gray-100` / `bg-blue-100` / `bg-green-100` のように背景色を変えてステータスを色分けしています。
- これは動作確認用の仮表示で、Tailwind が正しく効いているか（色・角丸・影が出るか）を目で確かめる役割です。

用語:「Flexbox（**flex**）」とは、子要素を横や縦に整列させるための CSS のレイアウト方式です。**space-x-2** のような間隔指定と組み合わせて、並びを手軽に整えられます。

9.2 開発サーバーの起動

ターミナルで以下のコマンドを実行します。

```
# npm run dev: package.json の "scripts" にある "dev" を実行
# 中身は「next dev --turboPack」で、Next.js の開発サーバーが起動する
# ファイルを保存するたびに自動でブラウザが更新される（ホットリロード）
# 停止するには Ctrl+C を押す
npm run dev
```

以下のような出力が表示されます。

```
▲ Next.js 15.x.x (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.x.x:3000

✓ Starting...
✓ Ready in xxxms
```

9.3 ブラウザでの確認

ブラウザで **http://localhost:3000** を開くと、以下のように表示されます。

ヘッダー部分: - 画面最上部に白い背景のヘッダーバーが表示されます - 左側に「書籍管理アプリ」というテキストが太字で表示されます - 右側に「書籍一覧」というテキストリンクと「書籍を登録する」という青いボタンが表示されます - 「書籍一覧」リンクは現在のページなので、薄い青の背景でハイライトされています

メインコンテンツ部分: - ヘッダーの下に薄いグレー（**bg-gray-50**）の背景があります - 「書籍一覧」という大きな見出し（30px, 太字）が表示されます - その下に「登録された書籍がここに表示されます。」という説明文があります - 白い角丸のカード内に「セットアップ完了」というメッセージが表示されます - カードの下部に3つのステータスバッジ（「未読」はグレー、「読書中」は青、「読了」は緑）が横並びで表示されます

確認ポイント:

確認項目	期待される状態
ヘッダーが表示される	白い背景のバーが画面上部にある
Tailwind CSS が適用されている	テキストに色やサイズが適用され、カードに角丸やシャドウがある
フォント（Noto Sans JP）が適用されている	日本語テキストが美しいフォントで表示される
レスポンシブ対応	ブラウザの幅を狭めても、レイアウトが崩れない
ナビゲーションリンクが動作する	「書籍を登録する」ボタンをクリックすると <code>/books/new</code> に遷移する（まだページを作成していないため 404 が表示されますが、URL が変わることを確認）

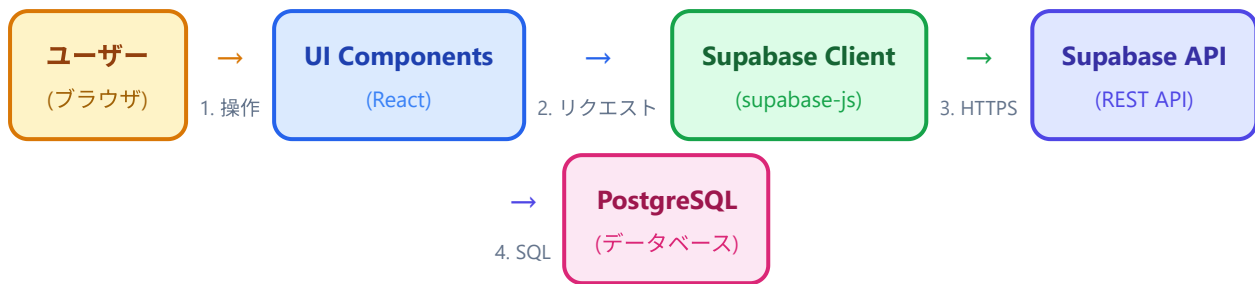
9.4 よくあるトラブルと対処法

症状	原因	対処法
<code>Module not found: Can't resolve '@/components/Header'</code>	ファイルが存在しない、またはパスが間違っている	<code>src/components/Header.tsx</code> が正しい場所に作成されているか確認してください。ファイル名の大文字小文字も一致している必要があります。
スタイルが全く適用されない	<code>globals.css</code> が正しく読み込まれていない	<code>src/app/layout.tsx</code> で <code>import "@/globals.css";</code> が記述されているか確認してください。
<code>環境変数 NEXT_PUBLIC_SUPABASE_URL</code> が設定されていません	<code>.env.local</code> が未作成または場所が間違っている	<code>.env.local</code> がプロジェクトのルートディレクトリ（ <code>package.json</code> と同じ階層）に存在するか確認してください。ファイル作成後は開発サーバーの再起動（ <code>Ctrl+C</code> で停止してから <code>npm run dev</code> ）が必要です。
ポート 3000 が既に使用されている	他のアプリが 3000 番を使っている	<code>npm run dev -- --port 3001</code> で別のポートを指定するか、使用中のアプリを停止してください。
日本語フォントが適用されていない	フォントの読み込みに失敗している	ネットワーク接続を確認してください。初回ビルド時に Google Fonts からフォントがダウンロードされます。

10. データフローの全体像

書籍管理アプリのデータの流れを図で確認しましょう。ユーザーの操作がどのようにデータベースに伝わり、結果がどのように画面に戻ってくるのかを理解することは、アプリ全体の設計を把握する上で非常に重要です。

10.1 全体アーキテクチャ



戻りの流れ: 5. クエリ結果 ← 6. JSON レスポンス ← 7. 型付きデータ (Book[]) ← 8. 画面更新 (再レンダリング)

10.2 各ステップの詳細解説

ステップ	場所	処理内容
1. ユーザーの操作	ブラウザ	ユーザーがボタンをクリックしたり、フォームに入力したりします。例: 「書籍を登録する」ボタンを押す。
2. コンポーネントからの呼び出し	UI Components	React コンポーネント内で Supabase クライアントの関数を呼び出します。例: <code>supabase.from("books").insert(newBook)</code>
3. HTTP リクエスト	Supabase Client	<code>@supabase/supabase-js</code> ライブラリが、JavaScript のコードを HTTPS リクエストに変換して Supabase のサーバーに送信します。URL や Anon Key は <code>.env.local</code> の環境変数から読み取られます。
4. SQL クエリの実行	Supabase API	Supabase の REST API (PostgREST) がリクエストを受け取り、対応する SQL クエリに変換して PostgreSQL データベースに実行します。RLS (行レベルセキュリティ) によるアクセス制御もこのタイミングで適用されます。
5. クエリ結果	PostgreSQL	データベースがクエリを実行し、結果を返します。
6. JSON レスポンス	Supabase API	クエリ結果を JSON 形式に変換し、クライアントにレスポンスとして返します。
7. 型付きデータ	Supabase Client	JSON レスポンスを受け取り、TypeScript の型 (<code>Book[]</code> など) に変換します。
8. 画面更新	UI Components → ブラウザ	受け取ったデータで React の state を更新し、画面が再レンダリングされます。ユーザーは最新のデータを画面上で確認できます。

10.3 具体例: 書籍一覧の取得

実際のコードでデータがどう流れるか、具体例で見てみましょう (このコードは次の章で実装します)。

▼このコードがやること（先に日本語で）：データベースから書籍の一覧を取り出す処理を、具体的なコードで追いかけます。さきほど作った Supabase クライアントを使い、「books テーブルから全件を、新しい順に取得する」という指示を出しています。`await`（待つ）は通信が終わるまで結果を待つための合図で、戻り値を `data`（成功データ）と `error`（エラー）に分けて受け取り、必ずエラーの有無を確認するのが安全な書き方です。この1行の裏で、前の図の通信ステップが自動で実行されています。

```
// 書籍一覧を取得するコードの流れ

// 1. Supabase クライアントをインポート
// named import で supabase 変数だけを取り込む (lib/supabase.ts で export const supabase したもの)
import { supabase } from "@lib/supabase";
// Book 型もインポート (型情報のみ。ランタイムには影響なし)
import { Book } from "@types/book";

// 2. データベースから書籍一覧を取得
// await: Promise (非同期処理の結果) を待つキーワード。async 関数内でのみ使用可能
// 戻り値は { data, error } のオブジェクトなので、分割代入で取り出す
const { data, error } = await supabase
  // books テーブルを指定 (SQL の FROM books に相当)
  .from("books")
  // すべてのカラムを取得 (SQL の SELECT * に相当)
  .select("*")
  // 作成日の降順 (新しい順) に並び替え (SQL の ORDER BY に相当)
  .order("created_at", {
    // false で降順 (新しい順)。true なら昇順 (古い順)
    ascending: false
  });

// 3. エラーハンドリング
// if (error): error が null/undefined でなければ true (エラーが発生した)
if (error) {
  // console.error: 開発者ツールのコンソールにエラーログを赤色で出力
  console.error("書籍の取得に失敗しました:", error.message);
  // ここで関数を抜ける
  return;
}

// 4. data は Book[] 型 (書籍の配列)
// data の型は自動で Book[] と推論されるが、明示的に型注釈を付けることもできる
const books: Book[] = data;
```

この1つの `supabase.from("books").select("*")` の呼び出しの裏側で、上の図の 2 → 3 → 4 → 5 → 6 → 7 のステップがすべて自動的に実行されています。Supabase クライアントがこれらの複雑な処理を抽象化してくれるため、開発者は SQL を直接書くことなく、シンプルな JavaScript/TypeScript のコードでデータベースを操作できます。

▼ コードを1つずつ分解して解説

この取得処理を、初心者がつまづきやすい部分ごとに1つずついねいに見ていきましょう。

解説1: 共有クライアントと型をインポートする

```
import { supabase } from "@lib/supabase";
import { Book } from "@types/book";
```

- 1行目で、さきほど `lib/supabase.ts` で作って `export` した `supabase`（データベースの窓口）を取り込んでいます。これで通信の準備は完了です。
- 2行目で `Book` 型を取り込み、取得したデータが「書籍の形」をしていることを TypeScript に伝えられるようにします。
- どちらも `@/`（= `src/`）から始まるパスエイリアスを使い、相対パスより読みやすく書いています。

用語:「型のインポート」とは、`Book` のような型定義を別ファイルから取り込むことです。データの形を共有することで、プロジェクト全体で同じ構造を保てます。

解説2: `await` で取得結果を待ち、分割代入で受け取る

```
const { data, error } = await supabase
  .from("books")
  .select("*")
  .order("created_at", {
    ascending: false
  });
```

- `await`（アウェイト）は、通信が終わるまで結果を待つための合図です。データベースとのやり取りには時間がかかるため、答えが返ってくるまでここで待機します。
- `.from("books")` で対象テーブルを、`.select("*")` で全カラムを、`.order("created_at", { ascending: false })` で「作成日の新しい順」を指定しています。SQL の `FROM / SELECT * / ORDER BY` に対応します。
- 戻り値は `{ data, error }` という形のオブジェクトなので、`const { data, error } = ...`（分割代入）で成功データとエラーを同時に取り出しています。

用語:「`await` / 非同期処理」とは、時間のかかる処理（通信など）の完了を待つ仕組みです。待っている間ほかの処理を止めず、結果が来たら続きを実行します。

```
if (error) {  
  console.error("書籍の取得に失敗しました:", error.message);  
  return;  
}  
  
const books: Book[] = data;
```

- `if (error)` は「エラーが入っていれば（通信が失敗していれば）」という意味です。Supabase はエラーを例外で投げる `error` に入れて返すため、自分でこのチェックをするのが安全な書き方です。
- 失敗時は `console.error(...)` で原因をコンソールに出し、`return` で処理を中断します。失敗したのに先へ進んでしまう事故を防げます。
- 問題がなければ、`data` を `Book[]`（書籍の配列）として受け取ります。`[]` は「その型がいくつも並んだ配列」を表します。

用語:「エラーハンドリング」とは、失敗が起きたときの対処をあらかじめ書いておくことです。通信は必ず成功するとは限らないため、`error` の確認はセットで行います。

まとめ

この章で実施した内容を振り返ります。

項目	内容
Next.js プロジェクトの作成	<code>create-next-app</code> で TypeScript + Tailwind CSS + App Router のプロジェクトを作成しました。
Tailwind CSS の基礎	ユーティリティファーストの概念と、よく使うクラスを学びました。
パッケージのインストール	<code>@supabase/supabase-js</code> をインストールしました。
ディレクトリ構成の設計	<code>app/</code> （ページ）、 <code>components/</code> （UI部品）、 <code>lib/</code> （ユーティリティ）、 <code>types/</code> （型定義）の4つのディレクトリで整理する構成にしました。
Supabase クライアントの設定	<code>.env.local</code> に環境変数を設定し、 <code>lib/supabase.ts</code> でクライアントを初期化しました。
型定義の作成	<code>Book</code> 、 <code>BookInsert</code> 、 <code>BookUpdate</code> の3つの型と、ステータス関連の定数を定義しました。
ルートレイアウトの設定	日本語フォント、メタデータ、全ページ共通のレイアウト構造を設定しました。
ヘッダーコンポーネントの作成	ナビゲーション付きの共通ヘッダーを作成しました。
動作確認	開発サーバーを起動し、ブラウザで表示を確認しました。

作成したファイル一覧:

src/		
├─ app/		
│ ├─ layout.tsx	✓	作成済み
│ └─ page.tsx	✓	作成済み（仮の内容）
│ └─ globals.css	✓	確認済み
├─ components/		
│ └─ Header.tsx	✓	作成済み
├─ lib/		
│ └─ supabase.ts	✓	作成済み
└─ types/		
└─ book.ts	✓	作成済み

次の章では、書籍一覧の表示機能を実装します。`BookCard`、`BookList`、`StatusBadge`、`RatingStars`、`LoadingSpinner` の各コンポーネントを作成し、Supabase からデータを取得してトップページに表示します。